源文件范围：run.py:1 → app/main.py:49

```
"""Start the local application without a JavaScript build toolchain."""
import argparse
from pathlib import Path
import os
import uvicorn
def main():
    parser = argparse.ArgumentParser(description="MedLabel Review 本地说明书审核工
作台")
    parser.add_argument("--host", default="127.0.0.1")
    parser.add_argument("--port", type=int, default=8000)
    parser.add_argument("--data-dir", default=str(Path(__file__).parent / "data"))
    args = parser.parse_args()
    os.environ["MEDLABEL_DATA_DIR"] = str(Path(args.data_dir).resolve())
    uvicorn.run("app.main:app", host=args.host, port=args.port)
if __name__ == "__main__":
    main()
"""MedLabel Review application."""
"""Local-first drug-label comparison and review application."""
from datetime import date
import hashlib
import json
import os
from pathlib import Path
from typing import Literal
from urllib.parse import urlsplit
from uuid import uuid4
from fastapi import FastAPI, File, Form, HTTPException, Request, UploadFile
from fastapi.exceptions import RequestValidationError
from fastapi.responses import FileResponse, JSONResponse, Response
from fastapi.staticfiles import StaticFiles
from pydantic import BaseModel, ConfigDict, Field
from app.comparison import compare_sections
from app.database import Database, audit, encode, now
from app.demo import DRUG, NEW_TEXT, OLD_TEXT
from app.documents import extract_text, normalize_sections, parse_sections
from app.reports import export_report
MAX_FILE_SIZE = 10 * 1024 * 1024
MAX_TEXT = 300_000
STATIC_DIR = Path(__file__).parent / "static"
class Input(BaseModel):
    model_config = ConfigDict(extra="forbid", str_strip_whitespace=True)
class DrugInput(Input):
    name: str = Field(min_length=1, max_length=200)
    manufacturer: str = Field(default="", max_length=300)
    specification: str = Field(default="", max_length=300)
    approval_number: str = Field(default="", max_length=200)
    notes: str = Field(default="", max_length=3000)
class TextVersionInput(Input):
    label: str = Field(min_length=1, max_length=120)
    version_date: date
    source: str = Field(default="", max_length=2000)
    text: str = Field(min_length=1, max_length=MAX_TEXT)
class SectionInput(Input):
    title: str = Field(min_length=1, max_length=200)
    text: str = Field(max_length=MAX_TEXT)
```

源文件范围：app/main.py:52 → app/main.py:114

```
00056  class SectionsInput(Input):
00057      sections: list[SectionInput] = Field(min_length=1, max_length=200)
00058  class ComparisonInput(Input):
00059      old_version_id: int = Field(gt=0)
00060      new_version_id: int = Field(gt=0)
00061  class ReviewInput(Input):
00062      status: Literal["pending", "confirmed", "ignored"]
00063      note: str = Field(default="", max_length=3000)
00064      revision: int = Field(ge=0)
00065  def required_row(connection, table, entity_id):
00066      # Table names are exclusively supplied by internal call sites.
00067      if table not in {"drugs", "versions", "changes", "comparisons"}:
00068          raise ValueError("Unknown table")
00069      row = connection.execute(f"SELECT * FROM {table} WHERE id=?", (entity_id,)).fe
00070  tchone()
00071      if row is None:
00072          raise HTTPException(404, "记录不存在")
00073      return row
00074  def version_data(row, full=False):
00075      result = dict(row)
00076      sections = json.loads(result.pop("sections"))
00077      result.pop("storage_name", None)
00078      result["warnings"] = json.loads(result["warnings"])
00079      result["section_count"] = len(sections)
00080      if full:
00081          result["sections"] = sections
00082      else:
00083          result.pop("text", None)
00084      return result
00085  def change_data(row):
00086      result = json.loads(row["payload"])
00087      result.update({key: row[key] for key in ("id", "comparison_id", "status", "not
00088  e", "reviewer", "reviewed_at", "revision")})
00089      return result
00090  def comparison_data(connection, entity_id, full=True):
00091      row = connection.execute(
00092          """SELECT c.*, d.name AS drug_name, ov.label AS old_label, nv.label AS new
00093  _label
00094          FROM comparisons c JOIN drugs d ON d.id=c.drug_id
00095          JOIN versions ov ON ov.id=c.old_version_id
00096          JOIN versions nv ON nv.id=c.new_version_id WHERE c.id=?""", (entity_id,)
00097      ).fetchone()
00098      if row is None:
00099          raise HTTPException(404, "比对记录不存在")
00100      result = dict(row)
00101      result["warnings"] = json.loads(result["warnings"])
00102      changes = [change_data(item) for item in connection.execute(
00103          "SELECT * FROM changes WHERE comparison_id=? ORDER BY id", (entity_id,)
00104      )]
00105      result["total_changes"] = len(changes)
00106      result["pending_changes"] = sum(item["status"] == "pending" for item in change
00107  s)
00108      result["status"] = "pending" if result["pending_changes"] else "completed"
00109      if full:
00110          result["changes"] = changes
```

源文件范围：app/main.py:115 → app/main.py:165

```
00111          result["old_version"] = version_data(required_row(connection, "versions",
00112  result["old_version_id"]), True)
00113          result["new_version"] = version_data(required_row(connection, "versions",
00114  result["new_version_id"]), True)
00115      return result
00116  def add_drug(connection, payload, actor):
00117      cursor = connection.execute(
00118          "INSERT INTO drugs(name,manufacturer,specification,approval_number,notes,c
00119  reated_at) VALUES(?,?,?,?,?,?)",
00120          (payload["name"], payload.get("manufacturer", ""), payload.get("specificat
00121  ion", ""),
00122           payload.get("approval_number", ""), payload.get("notes", ""), now()),
00123      )
00124      audit(connection, actor, "创建药品档案", "drug", cursor.lastrowid, {"name": pa
00125  yload["name"]})
00126      return cursor.lastrowid
00127  def add_version(connection, db, drug_id, label, version_date, source, filename, da
00128  ta, actor, created_files):
00129      required_row(connection, "drugs", drug_id)
00130      if not label.strip() or len(label.strip()) > 120:
00131          raise HTTPException(422, "版本名称须为 1–120 个字符")
00132      if len(source) > 2000:
00133          raise HTTPException(422, "来源信息不能超过 2000 个字符")
00134      if len(data) > MAX_FILE_SIZE:
00135          raise HTTPException(413, "文件不能超过 10 MB")
00136      if not data:
00137          raise HTTPException(422, "文件不能为空")
00138      try:
00139          parsed_date = date.fromisoformat(str(version_date)).isoformat()
00140      except (TypeError, ValueError):
00141          raise HTTPException(422, "版本日期格式应为 YYYY-MM-DD")
00142      clean_filename = Path(filename.replace("\\", "/")).name[:200]
00143      if not clean_filename:
00144          raise HTTPException(422, "文件名不能为空")
00145      try:
00146          text = extract_text(clean_filename, data)
00147          if len(text) > MAX_TEXT:
00148              raise ValueError("文档提取后不能超过 300000 个字符，请拆分文档")
00149          sections = parse_sections(text)
00150      except ValueError as error:
00151          raise HTTPException(422, str(error))
00152      warnings = []
00153      if len(sections) == 1:
00154          warnings.append("仅识别到一个章节，请在开始比对前核对章节划分。")
00155      storage_name = uuid4().hex + Path(clean_filename).suffix.lower()
00156      path = db.uploads / storage_name
00157      path.write_bytes(data)
00158      created_files.append(path)
00159      digest = hashlib.sha256(data).hexdigest()
00160      cursor = connection.execute(
00161          """INSERT INTO versions(drug_id,label,version_date,source,filename,sha256,
00162  storage_name,
00163          text,sections,warnings,created_at) VALUES(?,?,?,?,?,?,?,?,?,?)""",
00164          (drug_id, label.strip(), parsed_date, source.strip(), clean_filename, dige
00165  st, storage_name,
```

源文件范围：app/main.py:166 → app/main.py:216

```
00166           text, encode(sections), encode(warnings), now()),
00167       )
00168       audit(connection, actor, "导入说明书版本", "version", cursor.lastrowid,
00169             {"drug_id": drug_id, "label": label.strip(), "filename": clean_filename,
00170    "sha256": digest})
00171       return cursor.lastrowid
00172   def add_comparison(connection, old_id, new_id, actor):
00173       if old_id == new_id:
00174           raise HTTPException(422, "请选择两个不同版本")
00175       old = required_row(connection, "versions", old_id)
00176       new = required_row(connection, "versions", new_id)
00177       if old["drug_id"] != new["drug_id"]:
00178           raise HTTPException(422, "只能比对同一药品档案下的版本")
00179       if old["version_date"] > new["version_date"]:
00180           raise HTTPException(422, "旧版本日期不能晚于新版本日期")
00181       try:
00182           diff = compare_sections(json.loads(old["sections"]), json.loads(new["secti
00183   ons"]))
00184       except ValueError as error:
00185           raise HTTPException(422, str(error))
00186       cursor = connection.execute(
00187           """INSERT INTO comparisons(drug_id,old_version_id,new_version_id,unchanged
00188   _sections,warnings,created_at)
00189           VALUES(?,?,?,?,?,?)""", (old["drug_id"], old_id, new_id, diff["unchanged_s
00190   ections"], encode(diff.get("warnings", [])), now())
00191       )
00192       comparison_id = cursor.lastrowid
00193       for change in diff["changes"]:
00194           connection.execute("INSERT INTO changes(comparison_id,payload) VALUES(?,?)
00195   ", (comparison_id, encode(change)))
00196       audit(connection, actor, "创建版本比对", "comparison", comparison_id,
00197             {"old_version_id": old_id, "new_version_id": new_id, "changes": len(diff
00198   ["changes"])})
00199       return comparison_id
00200   def create_app(data_dir=None):
00201       application = FastAPI(
00202           title="药品说明书版本比对与变更审核系统", version="1.0.0",
00203           description="本地单操作员版本比对工具。输出用于人工复核，不判断医学正确性
00204   。",
00205       )
00206       db = Database(data_dir or os.environ.get("MEDLABEL_DATA_DIR", "data"))
00207       actor = os.environ.get("MEDLABEL_REVIEWER", "本地审核员").strip()[:200] or "本
00208   地审核员"
00209       allowed_hosts = {"localhost", "127.0.0.1", "::1"}
00210       # TestClient's synthetic hostname is only enabled for explicit app factories.
00211       if data_dir is not None:
00212           allowed_hosts.add("testserver")
00213       allowed_hosts.update(host.strip().lower() for host in os.environ.get("MEDLABEL
00214   _ALLOWED_HOSTS", "").split(",") if host.strip())
00215       application.state.db = db
00216       application.state.reviewer = actor
00217       @application.middleware("http")
00218       async def guard_requests(request: Request, call_next):
00219           # DNS rebinding must not make an attacker's origin a trusted local UI.
00220           host = request.headers.get("host", "")
```

源文件范围：app/main.py:217 → app/main.py:260

```
        try:
            authority = urlsplit("//" + host)
            host_port = authority.port or (443 if request.url.scheme == "https" el
se 80)
            if authority.username or authority.password or authority.path or autho
rity.query or authority.fragment:
                raise ValueError("Invalid authority")
            if (authority.hostname or "").lower() not in allowed_hosts:
                return JSONResponse({"detail": "不允许的访问主机，请使用 localhost
 或 127.0.0.1"}, status_code=400)
        except ValueError:
            return JSONResponse({"detail": "无效的访问主机"}, status_code=400)
        if request.method in {"POST", "PUT", "PATCH", "DELETE"}:
            if request.headers.get("X-Requested-With") != "MedLabelReview":
                return JSONResponse({"detail": "请求缺少本地应用校验头"}, status_c
ode=403)
            origin = request.headers.get("origin")
            if origin:
                try:
                    parsed_origin = urlsplit(origin)
                    origin_port = parsed_origin.port or (443 if parsed_origin.sche
me == "https" else 80)
                    same_origin = (parsed_origin.scheme == request.url.scheme
                                   and parsed_origin.hostname == authority.hostnam
e
                                   and origin_port == host_port
                                   and not parsed_origin.username and not parsed_o
rigin.password
                                   and not parsed_origin.path and not parsed_origi
n.query and not parsed_origin.fragment)
                except ValueError:
                    same_origin = False
                if not same_origin:
                    return JSONResponse({"detail": "不允许跨来源修改本地数据"}, st
atus_code=403)
            length = request.headers.get("content-length")
            if length:
                try:
                    if int(length) > MAX_FILE_SIZE + 256 * 1024:
                        return JSONResponse({"detail": "请求内容过大，文件不能超过
 10 MB"}, status_code=413)
                except ValueError:
                    return JSONResponse({"detail": "无效的请求长度"}, status_code=
400)
        response = await call_next(request)
        response.headers["X-Content-Type-Options"] = "nosniff"
        response.headers["X-Frame-Options"] = "DENY"
        response.headers["Referrer-Policy"] = "same-origin"
        response.headers["Cache-Control"] = "no-store"
        return response
    @application.exception_handler(RequestValidationError)
    async def validation_error(request, error):
        fields = [".".join(str(part) for part in item["loc"]) for item in error.er
rors()]
        return JSONResponse({"detail": "输入格式不正确，请检查：" + "、".join(fiel
```

源文件范围：app/main.py:260 → app/main.py:311

```
ds)}, status_code=422)
    @application.get("/api/health")
    def health():
        return {"status": "ok", "version": "1.0.0", "mode": "local-single-operator
", "reviewer": actor}
    @application.get("/api/dashboard")
    def dashboard():
        with db.connect() as connection:
            result = {table: connection.execute(f"SELECT COUNT(*) FROM {table}").f
etchone()[0]
                      for table in ("drugs", "versions", "comparisons")}
            result["pending_changes"] = connection.execute("SELECT COUNT(*) FROM c
hanges WHERE status='pending'").fetchone()[0]
            result["recent_activity"] = [dict(row) for row in connection.execute("
SELECT * FROM audit ORDER BY id DESC LIMIT 8")]
            return result
    @application.get("/api/drugs")
    def list_drugs():
        with db.connect() as connection:
            return [dict(row) for row in connection.execute(
                "SELECT d.*, (SELECT COUNT(*) FROM versions v WHERE v.drug_id=d.id
) AS version_count FROM drugs d ORDER BY d.id DESC")]
    @application.post("/api/drugs", status_code=201)
    def create_drug(payload: DrugInput):
        with db.connect(write=True) as connection:
            entity_id = add_drug(connection, payload.model_dump(), actor)
            result = dict(required_row(connection, "drugs", entity_id))
            result["version_count"] = 0
            return result
    @application.get("/api/drugs/{drug_id}")
    def get_drug(drug_id: int):
        with db.connect() as connection:
            result = dict(required_row(connection, "drugs", drug_id))
            result["version_count"] = connection.execute("SELECT COUNT(*) FROM ver
sions WHERE drug_id=?", (drug_id,)).fetchone()[0]
            return result
    @application.get("/api/drugs/{drug_id}/versions")
    def list_versions(drug_id: int):
        with db.connect() as connection:
            required_row(connection, "drugs", drug_id)
            return [version_data(row) for row in connection.execute(
                "SELECT * FROM versions WHERE drug_id=? ORDER BY version_date DESC
,id DESC", (drug_id,))]
    @application.post("/api/drugs/{drug_id}/versions", status_code=201)
    async def upload_version(drug_id: int, file: UploadFile = File(...), label: st
r = Form(...),
                             version_date: str = Form(...), source: str = Form("")
):
        created_files = []
        try:
            data = await file.read(MAX_FILE_SIZE + 1)
            with db.connect(write=True) as connection:
                entity_id = add_version(connection, db, drug_id, label, version_da
te, source,
                                        file.filename or "", data, actor, created_
```

源文件范围：app/main.py:311 → app/main.py:359

```
files)
                return version_data(required_row(connection, "versions", entity_id
), True)
        except Exception:
            for path in created_files:
                path.unlink(missing_ok=True)
            raise
        finally:
            await file.close()
    @application.post("/api/drugs/{drug_id}/versions/text", status_code=201)
    def text_version(drug_id: int, payload: TextVersionInput):
        created_files = []
        try:
            with db.connect(write=True) as connection:
                entity_id = add_version(connection, db, drug_id, payload.label, pa
yload.version_date,
                                        payload.source, "pasted-text.txt", payload
.text.encode("utf-8"), actor, created_files)
                return version_data(required_row(connection, "versions", entity_id
), True)
        except Exception:
            for path in created_files:
                path.unlink(missing_ok=True)
            raise
    @application.get("/api/versions/{version_id}")
    def get_version(version_id: int):
        with db.connect() as connection:
            return version_data(required_row(connection, "versions", version_id),
True)
    @application.get("/api/versions/{version_id}/file")
    def download_version(version_id: int):
        with db.connect() as connection:
            row = required_row(connection, "versions", version_id)
            path = (db.uploads / row["storage_name"]).resolve()
            if path.parent != db.uploads.resolve() or not path.is_file():
                raise HTTPException(404, "原始文件不存在")
            return FileResponse(path, filename=row["filename"], media_type="applic
ation/octet-stream")
    @application.put("/api/versions/{version_id}/sections")
    def update_sections(version_id: int, payload: SectionsInput):
        if sum(len(item.title) + len(item.text) for item in payload.sections) > MA
X_TEXT:
            raise HTTPException(422, "章节总长度不能超过 300000 个字符")
        with db.connect(write=True) as connection:
            row = required_row(connection, "versions", version_id)
            linked = connection.execute("SELECT id FROM comparisons WHERE old_vers
ion_id=? OR new_version_id=? LIMIT 1",
                                        (version_id, version_id)).fetchone()
            if linked:
                raise HTTPException(409, "此版本已用于比对，不能修改章节。请导入一
个新版本进行校正。")
            try:
                sections = normalize_sections([item.model_dump() for item in paylo
ad.sections])
            except ValueError as error:
```

源文件范围：app/main.py:360 → app/main.py:408

```
                raise HTTPException(422, str(error))
            for item in sections:
                item["start_line"] = None
                item["end_line"] = None
            warnings = ["章节经人工校正；原始文件和提取原文保持不变，章节位置不再等同于原文行号。"]
            connection.execute("UPDATE versions SET sections=?,warnings=? WHERE id=?", (encode(sections), encode(warnings), version_id))
            audit(connection, actor, "校正章节", "version", version_id,
                  {"before": json.loads(row["sections"]), "after": sections})
            return version_data(required_row(connection, "versions", version_id), True)
    @application.get("/api/comparisons")
    def list_comparisons():
        with db.connect() as connection:
            ids = [row[0] for row in connection.execute("SELECT id FROM comparisons ORDER BY id DESC")]
            return [comparison_data(connection, entity_id, False) for entity_id in ids]
    @application.post("/api/comparisons", status_code=201)
    def create_comparison(payload: ComparisonInput):
        with db.connect(write=True) as connection:
            entity_id = add_comparison(connection, payload.old_version_id, payload.new_version_id, actor)
            return comparison_data(connection, entity_id)
    @application.get("/api/comparisons/{comparison_id}")
    def get_comparison(comparison_id: int):
        with db.connect() as connection:
            return comparison_data(connection, comparison_id)
    @application.patch("/api/changes/{change_id}")
    def review_change(change_id: int, payload: ReviewInput):
        if payload.status == "ignored" and not payload.note:
            raise HTTPException(422, "忽略差异时必须填写原因")
        with db.connect(write=True) as connection:
            current = required_row(connection, "changes", change_id)
            if current["revision"] != payload.revision:
                raise HTTPException(409, "此差异已被更新，请刷新后重新审核")
            timestamp = now()
            connection.execute(
                "UPDATE changes SET status=?,note=?,reviewer=?,reviewed_at=?,revision=revision+1 WHERE id=? AND revision=?",
                (payload.status, payload.note, actor, timestamp, change_id, payload.revision),
            )
            audit(connection, actor, "审核变更", "change", change_id,
                  {"comparison_id": current["comparison_id"], "before": {"status": current["status"], "note": current["note"]},
                   "after": {"status": payload.status, "note": payload.note}, "revision": payload.revision + 1})
            return change_data(required_row(connection, "changes", change_id))
    @application.get("/api/audit")
    def list_audit():
        with db.connect() as connection:
            return [dict(row) for row in connection.execute("SELECT * FROM audit ORDER BY id DESC LIMIT 500")]
```

源文件范围：app/main.py:410 → app/database.py:14

```
    @application.get("/api/comparisons/{comparison_id}/export")
    def export_comparison(comparison_id: int, format: Literal["html", "csv", "json
"] = "html"):
        with db.connect() as connection:
            comparison = comparison_data(connection, comparison_id)
        body, media_type, filename = export_report(comparison, format)
        return Response(body, media_type=media_type, headers={"Content-Disposition
": f'attachment; filename="{filename}"'})
    @application.post("/api/demo")
    def seed_demo():
        created_files = []
        try:
            with db.connect(write=True) as connection:
                seeded = connection.execute("SELECT value FROM settings WHERE key=
'demo'").fetchone()
                if seeded:
                    return json.loads(seeded[0])
                drug_id = add_drug(connection, DRUG, actor)
                old_id = add_version(connection, db, drug_id, "模拟初始版", "2026-
01-01", "项目内置虚构示例，不用于临床",
                                     "demo-v1.txt", OLD_TEXT.encode("utf-8"), acto
r, created_files)
                new_id = add_version(connection, db, drug_id, "模拟修订版", "2026-
06-01", "项目内置虚构示例，不用于临床",
                                     "demo-v2.txt", NEW_TEXT.encode("utf-8"), acto
r, created_files)
                comparison_id = add_comparison(connection, old_id, new_id, actor)
                result = {"drug_id": drug_id, "comparison_id": comparison_id}
                connection.execute("INSERT INTO settings(key,value) VALUES('demo',
?)", (encode(result),))
                return result
        except Exception:
            for path in created_files:
                path.unlink(missing_ok=True)
            raise
    @application.get("/", include_in_schema=False)
    def index():
        path = STATIC_DIR / "index.html"
        if not path.exists():
            raise HTTPException(503, "界面文件尚未安装")
        return FileResponse(path)
    application.mount("/static", StaticFiles(directory=STATIC_DIR, check_dir=False
), name="static")
    return application
app = create_app()
"""SQLite persistence; every mutation and its audit record share a transaction."""
from contextlib import contextmanager
from datetime import datetime, timezone
import json
from pathlib import Path
import sqlite3
def now():
    return datetime.now(timezone.utc).isoformat(timespec="microseconds")
def encode(value):
    return json.dumps(value, ensure_ascii=False)
```

源文件范围：app/database.py:17 → app/database.py:71

```
SCHEMA = """
CREATE TABLE IF NOT EXISTS drugs (
 id INTEGER PRIMARY KEY, name TEXT NOT NULL, manufacturer TEXT NOT NULL DEFAULT ''
,
 specification TEXT NOT NULL DEFAULT '', approval_number TEXT NOT NULL DEFAULT '',
 notes TEXT NOT NULL DEFAULT '', created_at TEXT NOT NULL
);
CREATE TABLE IF NOT EXISTS versions (
 id INTEGER PRIMARY KEY, drug_id INTEGER NOT NULL REFERENCES drugs(id),
 label TEXT NOT NULL, version_date TEXT NOT NULL, source TEXT NOT NULL DEFAULT '',
 filename TEXT NOT NULL, sha256 TEXT NOT NULL, storage_name TEXT NOT NULL,
 text TEXT NOT NULL, sections TEXT NOT NULL, warnings TEXT NOT NULL DEFAULT '[]',
 created_at TEXT NOT NULL
);
CREATE TABLE IF NOT EXISTS comparisons (
 id INTEGER PRIMARY KEY, drug_id INTEGER NOT NULL REFERENCES drugs(id),
 old_version_id INTEGER NOT NULL REFERENCES versions(id),
 new_version_id INTEGER NOT NULL REFERENCES versions(id),
 unchanged_sections INTEGER NOT NULL, warnings TEXT NOT NULL DEFAULT '[]',
 created_at TEXT NOT NULL,
 CHECK(old_version_id != new_version_id)
);
CREATE TABLE IF NOT EXISTS changes (
 id INTEGER PRIMARY KEY, comparison_id INTEGER NOT NULL REFERENCES comparisons(id)
,
 payload TEXT NOT NULL, status TEXT NOT NULL DEFAULT 'pending'
 CHECK(status IN ('pending','confirmed','ignored')),
 note TEXT NOT NULL DEFAULT '', reviewer TEXT NOT NULL DEFAULT '',
 reviewed_at TEXT, revision INTEGER NOT NULL DEFAULT 0
);
CREATE TABLE IF NOT EXISTS audit (
 id INTEGER PRIMARY KEY, action TEXT NOT NULL, entity_type TEXT NOT NULL,
 entity_id INTEGER NOT NULL, detail TEXT NOT NULL, actor TEXT NOT NULL,
 created_at TEXT NOT NULL
);
CREATE TABLE IF NOT EXISTS settings (key TEXT PRIMARY KEY, value TEXT NOT NULL);
CREATE INDEX IF NOT EXISTS versions_drug ON versions(drug_id);
CREATE INDEX IF NOT EXISTS changes_comparison ON changes(comparison_id);
CREATE INDEX IF NOT EXISTS comparisons_versions ON comparisons(old_version_id,new_
version_id);
"""
class Database:
    def __init__(self, data_dir):
        self.directory = Path(data_dir).resolve()
        self.directory.mkdir(parents=True, exist_ok=True)
        self.uploads = self.directory / "uploads"
        self.uploads.mkdir(exist_ok=True)
        self.path = self.directory / "medlabel.sqlite3"
        with self.connect() as connection:
            connection.executescript(SCHEMA)
    @contextmanager
    def connect(self, write=False):
        connection = sqlite3.connect(self.path, timeout=15)
        connection.row_factory = sqlite3.Row
        connection.execute("PRAGMA foreign_keys = ON")
```

源文件范围：app/database.py:72 → app/documents.py:43

```
00551          connection.execute("PRAGMA journal_mode = WAL")
00552          try:
00553              if write:
00554                  connection.execute("BEGIN IMMEDIATE")
00555              yield connection
00556              connection.commit()
00557          except Exception:
00558              connection.rollback()
00559              raise
00560          finally:
00561              connection.close()
00562  def audit(connection, actor, action, entity_type, entity_id, detail):
00563      connection.execute(
00564          "INSERT INTO audit(action,entity_type,entity_id,detail,actor,created_at) V
00565  ALUES(?,?,?,?,?,?)",
00566          (action, entity_type, entity_id, encode(detail) if not isinstance(detail,
00567  str) else detail, actor, now()),
00568      )
00569  """Extract and structure drug leaflets without inferring medical meaning.
00570  Only user supplied text is processed.  The original file/text is retained by the
00571  API; these sections are a reviewable representation, never a replacement source.
00572  """
00573  from __future__ import annotations
00574  import io
00575  import re
00576  import unicodedata
00577  import zipfile
00578  from collections import Counter
00579  from pathlib import PurePath
00580  MAX_FILE_BYTES = 10 * 1024 * 1024
00581  MAX_TEXT_CHARS = 1_000_000
00582  MAX_PDF_PAGES = 100
00583  MAX_SECTIONS = 500
00584  MAX_DOCX_EXPANDED = 40 * 1024 * 1024
00585  # Alias matching is deliberately conservative.  Similar clinical concepts must
00586  # not be merged simply because a fuzzy text matcher considers their names close.
00587  _ALIASES = {
00588      "药品名称": "药品名称", "药物名称": "药品名称",
00589      "成份": "成份", "成分": "成份", "主要成份": "成份", "主要成分": "成份",
00590      "性状": "性状", "适应症": "适应症", "适应证": "适应症",
00591      "功能主治": "功能主治", "规格": "规格", "用法用量": "用法用量",
00592      "用法与用量": "用法用量", "用法及用量": "用法用量",
00593      "不良反应": "不良反应", "禁忌": "禁忌", "禁忌症": "禁忌",
00594      "禁忌证": "禁忌", "注意事项": "注意事项", "警告": "警告",
00595      "警示语": "警告", "警示": "警告", "黑框警告": "黑框警告",
00596      "孕妇及哺乳期妇女用药": "孕妇及哺乳期妇女用药",
00597      "孕妇和哺乳期妇女用药": "孕妇及哺乳期妇女用药",
00598      "孕妇及哺乳期用药": "孕妇及哺乳期妇女用药",
00599      "儿童用药": "儿童用药", "老年用药": "老年用药", "老年患者用药": "老年用药",
00600      "特殊人群用药": "特殊人群用药", "药物相互作用": "药物相互作用",
00601      "药物过量": "药物过量", "临床试验": "临床试验",
00602      "药理毒理": "药理毒理", "药理作用": "药理作用", "毒理研究": "毒理研究",
00603      "药代动力学": "药代动力学", "贮藏": "贮藏", "储藏": "贮藏",
00604      "包装": "包装", "有效期": "有效期", "执行标准": "执行标准",
00605      "批准文号": "批准文号", "药品批准文号": "批准文号",
```

源文件范围：app/documents.py:44 → app/documents.py:102

```
00606      "上市许可持有人": "上市许可持有人", "药品上市许可持有人": "上市许可持有人",
00607      "生产企业": "生产企业", "生产厂家": "生产企业",
00608      "核准日期": "核准日期", "修改日期": "修改日期", "修订日期": "修改日期",
00609  }
00610  _BRACKET_HEADING = re.compile(r"^\s*(?:【([^】\n]{1,60})】|\[([^\]\n]{1,60})\]| [(
00611  [^］\n]{1,60})］)\s*[:：]?\s*(.*)$")
00612  _NUMBER_PREFIX = re.compile(r"^\s*(?:(?:[一二三四五六七八九十百]+|\d+)[、..．]\s*|[
00613  （(](?:[一二三四五六七八九十百]+|\d+)[）)]\s*)")
00614  def _clean_text(text: str) -> str:
00615      text = text.replace("\r\n", "\n").replace("\r", "\n").replace("\ufeff", "")
00616      if "\x00" in text:
00617          raise ValueError("文档包含二进制或空字符，请转换为文字版文件后重试")
00618      if len(text) > MAX_TEXT_CHARS:
00619          raise ValueError("文档文字超过 100 万字符，请拆分为单份药品说明书")
00620      if not text.strip():
00621          raise ValueError("未提取到可用文字；扫描件需要先进行 OCR 并人工核对")
00622      return text
00623  def _pdf_contains_image(page, reader) -> bool:
00624      """Inspect resource graphs and parsed operators, never text-density guesses.
00625      PDF images can coexist with a tiny text layer (for example only a page
00626      number), be nested in Form XObjects, or be embedded as inline images. V1
00627      deliberately rejects all such pages, including otherwise harmless logos.
00628      """
00629      from pypdf.generic import ContentStream
00630      seen_resources: set[int] = set()
00631      seen_streams: set[int] = set()
00632      def resolve(value):
00633          return value.get_object() if hasattr(value, "get_object") else value
00634      def inspect(stream, resources, depth=0):
00635          if depth > 32 or len(seen_resources) + len(seen_streams) > 2000:
00636              raise ValueError("PDF 图形结构过于复杂，无法确认文字完整性；请转换为完
00637  整文字并核对")
00638          stream = resolve(stream)
00639          resources = resolve(resources) if resources else {}
00640          if stream is not None and id(stream) not in seen_streams:
00641              seen_streams.add(id(stream))
00642              parsed = stream if isinstance(stream, ContentStream) else ContentStrea
00643  m(stream, reader)
00644              if any(operator == b"INLINE IMAGE" for _, operator in parsed.operation
00645  s):
00646                  return True
00647          if id(resources) in seen_resources:
00648              return False
00649          seen_resources.add(id(resources))
00650          for category in ("/XObject", "/Pattern"):
00651              objects = resolve(resources.get(category, {}))
00652              for reference in objects.values():
00653                  obj = resolve(reference)
00654                  if obj.get("/Subtype") == "/Image":
00655                      return True
00656                  if obj.get("/Subtype") == "/Form" or obj.get("/PatternType") == 1:
00657                      if inspect(obj, obj.get("/Resources", resources), depth + 1):
00658                          return True
00659          return False
00660      resources = page.get("/Resources", {})
```

源文件范围：app/documents.py:103 → app/documents.py:160

```
00661      if inspect(page.get_contents(), resources):
00662          return True
00663      # Annotation appearance streams may also contain images not referenced by
00664      # the main page content stream (for example a scanned stamp or attachment).
00665      for reference in resolve(page.get("/Annots", [])):
00666          annotation = resolve(reference)
00667          appearances = resolve(annotation.get("/AP", {}))
00668          for appearance in appearances.values():
00669              appearance = resolve(appearance)
00670              candidates = [appearance] if hasattr(appearance, "get_data") else [res
00671  olve(value) for value in appearance.values()]
00672              for candidate in candidates:
00673                  if inspect(candidate, candidate.get("/Resources", resources)):
00674                      return True
00675      return False
00676  def _pdf_text(data: bytes) -> str:
00677      from pypdf import PdfReader
00678      reader = PdfReader(io.BytesIO(data), strict=False)
00679      if reader.is_encrypted:
00680          raise ValueError("PDF 已加密，请提供解密后的文字版 PDF")
00681      if len(reader.pages) > MAX_PDF_PAGES:
00682          raise ValueError("PDF 超过 100 页，请拆分为单份药品说明书")
00683      texts: list[str] = []
00684      length = 0
00685      for index, page in enumerate(reader.pages, start=1):
00686          if _pdf_contains_image(page, reader):
00687              raise ValueError(f"此 PDF 含图片内容，可能遗漏文字；请先转换为完整文字
00688  并核对（第 {index} 页）")
00689          text = page.extract_text() or ""
00690          if text.strip():
00691              texts.append(text)
00692              length += len(text) + 2
00693          if length > MAX_TEXT_CHARS:
00694              raise ValueError("文档文字超过 100 万字符，请拆分为单份药品说明书")
00695      return "\n\n".join(texts)
00696  def _docx_text(data: bytes) -> str:
00697      from docx import Document
00698      from docx.oxml.ns import qn
00699      with zipfile.ZipFile(io.BytesIO(data)) as archive:
00700          members = archive.infolist()
00701          if len(members) > 2500 or sum(member.file_size for member in members) > MA
00702  X_DOCX_EXPANDED:
00703              raise ValueError("DOCX 解压后体积或文件数量过大，请简化文件后重试")
00704          if any(member.flag_bits & 1 for member in members):
00705              raise ValueError("不支持加密的 DOCX 文件")
00706          if any(member.file_size > 1024 * 1024 and member.file_size / max(member.co
00707  mpress_size, 1) > 200 for member in members):
00708              raise ValueError("DOCX 压缩比例异常，请重新保存为普通文档")
00709          if "word/document.xml" not in archive.namelist():
00710              raise ValueError("文件不是有效的 DOCX 文档")
00711      document = Document(io.BytesIO(data))
00712      headers, footers = [], []
00713      seen_parts: set[str] = set()
00714      for relation in document.part.rels.values():
00715          if relation.is_external or not relation.reltype.endswith(("/header", "/foo
```

源文件范围：app/documents.py:160 → app/documents.py:215

```
ter")):
                continue
            part = relation.target_part
            if str(part.partname) in seen_parts:
                continue
            seen_parts.add(str(part.partname))
            (headers if relation.reltype.endswith("/header") else footers).append(part
.element)
        roots = headers + [document.element.body] + footers
        for root in roots:
            if root.xpath(".//w:footnoteReference | .//w:endnoteReference"):
                raise ValueError("DOCX 含脚注或尾注，请先转换为包含全部文字的 TXT，避
免遗漏")
            if root.xpath(".//w:ins | .//w:del"):
                raise ValueError("DOCX 含未处理的修订记录，请确认修订后重新上传")
        # Iterate paragraphs in XML document order. This includes paragraphs inside
        # table cells and text boxes, which Document.paragraphs alone would omit.
        paragraphs: list[str] = []
        for root in roots:
            if root in headers:
                paragraphs.append("【页眉内容】")
            elif root in footers:
                paragraphs.append("【页脚内容】")
            for paragraph in root.iter(qn("w:p")):
                fragments: list[str] = []
                for node in paragraph.iter():
                    # Text-box paragraphs are visited in their own iteration; skip
                    # their text here to avoid duplication in the outer paragraph.
                    parent = node.getparent()
                    in_nested_paragraph = False
                    while parent is not None and parent is not paragraph:
                        if parent.tag == qn("w:p"):
                            in_nested_paragraph = True
                            break
                        parent = parent.getparent()
                    if in_nested_paragraph:
                        continue
                    if node.tag == qn("w:t") and node.text:
                        fragments.append(node.text)
                    elif node.tag == qn("w:tab"):
                        fragments.append("\t")
                    elif node.tag in {qn("w:br"), qn("w:cr")}:
                        fragments.append("\n")
                paragraphs.append("".join(fragments))
        return "\n".join(paragraphs)
    def extract_text(filename: str, data: bytes) -> str:
        """Extract a supported document or raise a user-facing ``ValueError``.
        Filenames are used only to select format, never as filesystem destinations.
        Size checks are duplicated here so callers other than HTTP cannot bypass them.
        """
        if not isinstance(data, bytes) or not data:
            raise ValueError("文件为空，请选择包含文字的文件")
        if len(data) > MAX_FILE_BYTES:
            raise ValueError("文件超过 10 MB，请压缩或拆分后重试")
        extension = PurePath(filename.replace("\\", "/")).suffix.lower()
```

源文件范围：app/documents.py:216 → app/documents.py:272

```
00771      try:
00772          if extension == ".txt":
00773              text = None
00774              for encoding in ("utf-8-sig", "gb18030"):
00775                  try:
00776                      text = data.decode(encoding)
00777                      break
00778                  except UnicodeDecodeError:
00779                      pass
00780              if text is None:
00781                  raise ValueError("TXT 编码无法识别，请另存为 UTF-8")
00782          elif extension == ".pdf":
00783              if not data.lstrip().startswith(b"%PDF-"):
00784                  raise ValueError("文件内容不是有效的 PDF")
00785              text = _pdf_text(data)
00786          elif extension == ".docx":
00787              text = _docx_text(data)
00788          else:
00789              raise ValueError("仅支持 TXT、文字版 PDF 和 DOCX 文件；旧版 DOC 请另存
00790  为 DOCX")
00791          return _clean_text(text)
00792      except ValueError:
00793          raise
00794      except Exception as exc:
00795          raise ValueError("文档无法解析，可能已损坏或格式不匹配；请重新导出为文字版
00796  文件") from exc
00797  def canonical_title(title: str) -> str:
00798      """Normalize only heading formatting and explicitly documented aliases."""
00799      normalized = unicodedata.normalize("NFKC", title).strip().strip("【】[] 〔〕::
00800   ")
00801      normalized = _NUMBER_PREFIX.sub("", normalized)
00802      compact = re.sub(r"\s+", "", normalized)
00803      return _ALIASES.get(compact, compact or "未命名章节")
00804  def _heading(line: str) -> tuple[str, str] | None:
00805      match = _BRACKET_HEADING.match(line)
00806      if match:
00807          title = next(part for part in match.groups()[:3] if part is not None).stri
00808  p()
00809          # [1] at the start of a reference paragraph is not a section title.
00810          if re.fullmatch(r"[\d\W_]+", title):
00811              return None
00812          return title, match.group(4)
00813      stripped = _NUMBER_PREFIX.sub("", line.strip())
00814      # Unbracketed headings must exactly match a known heading, or be followed
00815      # by a colon. Arbitrary short body sentences are never promoted to headings.
00816      title, separator, remainder = stripped.partition(": ")
00817      if not separator:
00818          title, separator, remainder = stripped.partition(":")
00819      compact = re.sub(r"\s+", "", title)
00820      if compact in _ALIASES:
00821          return title.strip(), remainder.strip() if separator else ""
00822      return None
00823  def _assign_keys(sections: list[dict]) -> list[dict]:
00824      counts: Counter[str] = Counter()
00825      for section in sections:
```

源文件范围：app/documents.py:273 → app/documents.py:334

```
00826          # Reserve ASCII '#' for the occurrence suffix; literal title characters
00827          # must not collide with generated keys such as 成份#2.
00828          canonical = canonical_title(section["title"]).replace("#", "＃")
00829          counts[canonical] += 1
00830          section["key"] = canonical if counts[canonical] == 1 else f"{canonical}#{c
00831  ounts[canonical]}"
00832      return sections
00833  def parse_sections(text: str) -> list[dict]:
00834      """Split sections, preserving preamble, duplicates, and source line ranges.
00835      Line numbers refer to extracted text, not PDF page layout. Unknown bracketed
00836      headings remain separate sections. Unstructured documents become one 全文
00837      section so content cannot silently disappear.
00838      """
00839      text = _clean_text(text)
00840      lines = text.split("\n")
00841      sections: list[dict] = []
00842      current: dict | None = None
00843      buffer: list[str] = []
00844      def flush(end_line: int) -> None:
00845          if current is None:
00846              return
00847          current["text"] = "\n".join(buffer).strip()
00848          current["end_line"] = max(current["start_line"], end_line)
00849          sections.append(current)
00850      for number, line in enumerate(lines, start=1):
00851          heading = _heading(line)
00852          if heading is not None:
00853              flush(number - 1)
00854              title, remainder = heading
00855              current = {"title": title, "start_line": number}
00856              buffer = [remainder] if remainder else []
00857          else:
00858              if current is None:
00859                  if not line.strip():
00860                      continue
00861                  current = {"title": "前言", "start_line": number}
00862              buffer.append(line)
00863      flush(len(lines))
00864      if len(sections) == 1 and sections[0]["title"] == "前言":
00865          sections[0]["title"] = "全文"
00866      if len(sections) > MAX_SECTIONS:
00867          raise ValueError("章节数量超过 500，请检查文件格式或拆分文档")
00868      return _assign_keys(sections)
00869  def normalize_sections(sections: list[dict]) -> list[dict]:
00870      """Validate a manual section correction and regenerate stable keys.
00871      Source line numbers are intentionally cleared: manual corrections cannot
00872      honestly claim exact correspondence with extraction line positions.
00873      """
00874      if not isinstance(sections, list) or not sections:
00875          raise ValueError("至少保留一个章节")
00876      if len(sections) > MAX_SECTIONS:
00877          raise ValueError("章节数量不能超过 500")
00878      result: list[dict] = []
00879      total = 0
00880      for section in sections:
```

源文件范围：app/documents.py:335 → app/comparison.py:47

```
00881          if not isinstance(section, dict):
00882              raise ValueError("章节格式错误")
00883          title = section.get("title")
00884          text = section.get("text")
00885          if not isinstance(title, str) or not title.strip() or len(title.strip()) >
00886   120:
00887              raise ValueError("章节标题应为 1 至 120 个字符")
00888          if not isinstance(text, str) or "\x00" in text:
00889              raise ValueError("章节内容应为有效文本")
00890          text = text.replace("\r\n", "\n").replace("\r", "\n").strip()
00891          total += len(text)
00892          result.append({"title": title.strip(), "text": text, "start_line": None, "
00893  end_line": None})
00894      if total > MAX_TEXT_CHARS:
00895          raise ValueError("章节文字总量不能超过 100 万字符")
00896      if not any(section["text"] for section in result):
00897          raise ValueError("章节内容不能全部为空")
00898      return _assign_keys(result)
00899  """Deterministic section alignment and review prioritization.
00900  Priority and flags identify textual review tasks, not clinical risk or a medical
00901  recommendation. Inline segments are raw text and MUST be escaped by renderers.
00902  """
00903  from __future__ import annotations
00904  import re
00905  from collections import Counter
00906  from difflib import SequenceMatcher
00907  from .documents import canonical_title
00908  _IMPORTANT_SECTIONS = {
00909      "用法用量", "禁忌", "不良反应", "注意事项", "警告", "黑框警告",
00910      "孕妇及哺乳期妇女用药", "儿童用药", "老年用药", "特殊人群用药",
00911      "药物相互作用", "药物过量", "适应症", "功能主治",
00912  }
00913  _NUMBER = re.compile(r"(?<![A-Za-z0-9.])[-+]?\d+(?:[.,]\d+)*(?:\s*[%%])?")
00914  _UNIT = re.compile(r"(?<![A-Za-z])(?:mcg|μg|µg|mg|kg|g|mL|ml|L|IU|mmol|mol|μL|µL|℃
00915  |°C|毫克|微克|克|毫升|升|单位|片|粒|支|袋|滴)(?![A-Za-z])", re.IGNORECASE)
00916  def _comparison_text(text: str) -> str:
00917      """Ignore layout whitespace without merging distinct numeric tokens.
00918      In particular '1 0 mg' never becomes '10 mg'; decimal points, units,
00919      punctuation and negation remain significant.
00920      """
00921      text = re.sub(r"\s+", " ", text).strip()
00922      text = re.sub(r"(?<=[\u3400-\u9fff]) +(?=[\u3400-\u9fff])", "", text)
00923      return text
00924  def _indexed(sections: list[dict]) -> tuple[dict[str, dict], list[str]]:
00925      counts: Counter[str] = Counter()
00926      result: dict[str, dict] = {}
00927      order: list[str] = []
00928      # Recompute keys from headings rather than trusting mutable caller keys.
00929      for section in sections:
00930          title = canonical_title(section["title"]).replace("#", "＃")
00931          counts[title] += 1
00932          key = title if counts[title] == 1 else f"{title}#{counts[title]}"
00933          result[key] = section
00934          order.append(key)
00935      return result, order
```

源文件范围：app/comparison.py:50 → app/comparison.py:104

```
def _location(section: dict | None) -> str:
    if section is None:
        return "无对应章节"
    start, end = section.get("start_line"), section.get("end_line")
    if not isinstance(start, int) or not isinstance(end, int):
        return "手工校正章节"
    return f"提取文本第 {start} 行" if start == end else f"提取文本第 {start}–{end
} 行"
def _inline_segments(old: str, new: str) -> tuple[list[dict], bool]:
    if not old:
        return ([{"op": "insert", "text": new}] if new else []), False
    if not new:
        return [{"op": "delete", "text": old}], False
    # Character-level matching is useful for Chinese units and negations, but
    # huge or pathological strings need a bounded whole-section fallback.
    if len(old) + len(new) > 30_000:
        return [{"op": "delete", "text": old}, {"op": "insert", "text": new}], Tru
e
    matcher = SequenceMatcher(None, old, new, autojunk=True)
    segments: list[dict] = []
    for op, a_start, a_end, b_start, b_end in matcher.get_opcodes():
        if op == "equal":
            segments.append({"op": "equal", "text": old[a_start:a_end]})
        else:
            if op in {"delete", "replace"}:
                segments.append({"op": "delete", "text": old[a_start:a_end]})
            if op in {"insert", "replace"}:
                segments.append({"op": "insert", "text": new[b_start:b_end]})
    return segments, False
def compare_sections(old_sections: list[dict], new_sections: list[dict]) -> dict:
    """Compare each canonical heading, pairing repeated headings by occurrence.
    Reordering alone is reported as a warning, not falsely presented as new
    clinical content. Differences are returned in new-document order, followed
    by removed sections in old-document order.
    """
    old_map, old_order = _indexed(old_sections)
    new_map, new_order = _indexed(new_sections)
    warnings: list[str] = []
    if any("#" in key for key in old_order + new_order):
        warnings.append("存在重复标题，已按同名章节出现顺序配对；请人工核对章节对
应关系。")
    if "全文" in old_map or "全文" in new_map:
        warnings.append("至少一个版本未识别出标准章节；可在创建比对前手工整理章节
，以提高定位精度。")
    shared_old = [key for key in old_order if key in new_map]
    shared_new = [key for key in new_order if key in old_map]
    if shared_old != shared_new:
        warnings.append("章节顺序发生变化；已按标题对齐，顺序调整本身不计为内容变
更。")
    changes: list[dict] = []
    unchanged = 0
    all_keys = new_order + [key for key in old_order if key not in new_map]
    for key in all_keys:
        old, new = old_map.get(key), new_map.get(key)
        old_text = old["text"] if old else ""
```

源文件范围：app/comparison.py:105 → app/static/styles.css:20

```
00991          new_text = new["text"] if new else ""
00992          if old is not None and new is not None and _comparison_text(old_text) == _
00993  comparison_text(new_text):
00994              unchanged += 1
00995              continue
00996          kind = "added" if old is None else "removed" if new is None else "modified
00997  "
00998          section = new if new is not None else old
00999          title = section["title"]
01000          canonical = canonical_title(title)
01001          flags: list[str] = []
01002          if canonical in _IMPORTANT_SECTIONS:
01003              flags.append("重点章节变更，需人工复核")
01004          if _NUMBER.findall(old_text) != _NUMBER.findall(new_text):
01005              flags.append("数字或比例变化")
01006          if _UNIT.findall(old_text) != _UNIT.findall(new_text):
01007              flags.append("单位或剂型计量词变化")
01008          segments, coarse = _inline_segments(old_text, new_text)
01009          if coarse:
01010              flags.append("长章节采用整段展示，请对照原文复核")
01011          changes.append({
01012              "section_key": key,
01013              "section_title": title,
01014              "kind": kind,
01015              "priority": "high" if canonical in _IMPORTANT_SECTIONS or any("变化" i
01016  n flag for flag in flags) else "normal",
01017              "old_text": old_text,
01018              "new_text": new_text,
01019              "old_location": _location(old),
01020              "new_location": _location(new),
01021              "segments": segments,
01022              "flags": flags,
01023          })
01024      return {"changes": changes, "unchanged_sections": unchanged, "warnings": warni
01025  ngs}
01026  :root {
01027    --ink: #203a38;
01028    --muted: #74817d;
01029    --line: #e1e7e1;
01030    --paper: #fff;
01031    --bg: #f5f6f2;
01032    --teal: #176b5b;
01033    --teal-soft: #eaf4ee;
01034    --nav: #143c38;
01035    --red: #aa4945;
01036    --red-soft: #fcecea;
01037    --green: #237251;
01038    --green-soft: #e6f3e9;
01039    --amber: #9a6a1f;
01040    --amber-soft: #fbf3df;
01041    --radius: 14px;
01042    --shadow: 0 14px 50px #183c3710;
01043    font-family:
01044      Inter,
01045      -apple-system,
```

源文件范围：app/static/styles.css:21 → app/static/styles.css:85

```
01046      BlinkMacSystemFont,
01047      "Segoe UI",
01048      "PingFang SC",
01049      "Hiragino Sans GB",
01050      "Microsoft YaHei",
01051      sans-serif;
01052    font-size: 14px;
01053    color: var(--ink);
01054    background: var(--bg);
01055    font-synthesis: none;
01056  }
01057  * {
01058    box-sizing: border-box;
01059  }
01060  body {
01061    margin: 0;
01062  }
01063  button,
01064  input,
01065  textarea,
01066  select {
01067    font: inherit;
01068  }
01069  button,
01070  a,
01071  input,
01072  textarea,
01073  select {
01074    -webkit-tap-highlight-color: transparent;
01075  }
01076  button,
01077  a {
01078    touch-action: manipulation;
01079  }
01080  button {
01081    cursor: pointer;
01082  }
01083  a {
01084    color: var(--teal);
01085    text-decoration: none;
01086  }
01087  a:hover {
01088    text-decoration: none;
01089    color: #0b5043;
01090  }
01091  button:disabled {
01092    opacity: 0.55;
01093    cursor: wait;
01094  }
01095  button:focus-visible,
01096  a:focus-visible,
01097  input:focus-visible,
01098  textarea:focus-visible,
01099  select:focus-visible {
01100    outline: 3px solid #6eb6a4;
```

源文件范围：app/static/styles.css:86 → app/static/styles.css:149

```
01101    outline-offset: 3px;
01102  }
01103  button,
01104  input,
01105  select,
01106  textarea {
01107    border-radius: 8px;
01108  }
01109  svg {
01110    display: inline-block;
01111    width: 20px;
01112    height: 20px;
01113    fill: none;
01114    stroke: currentColor;
01115    stroke-width: 1.7;
01116    stroke-linecap: round;
01117    stroke-linejoin: round;
01118    vertical-align: middle;
01119  }
01120  h1,
01121  h2,
01122  h3,
01123  p {
01124    margin-top: 0;
01125  }
01126  h1 {
01127    font-size: 27px;
01128    line-height: 1.4;
01129    letter-spacing: -0.6px;
01130    font-weight: 680;
01131    margin-bottom: 7px;
01132  }
01133  h2 {
01134    font-size: 18px;
01135    letter-spacing: -0.2px;
01136    margin-bottom: 5px;
01137  }
01138  h3 {
01139    font-size: 15px;
01140    margin-bottom: 7px;
01141  }
01142  p {
01143    line-height: 1.8;
01144  }
01145  button svg {
01146    width: 17px;
01147    height: 17px;
01148    flex: none;
01149  }
01150  .skip-link {
01151    position: fixed;
01152    top: -60px;
01153    left: 20px;
01154    padding: 10px;
01155    background: white;
```

源文件范围：app/static/styles.css:150 → app/static/styles.css:211

```
01156    z-index: 100;
01157  }
01158  .skip-link:focus {
01159    top: 15px;
01160  }
01161  .sidebar {
01162    position: fixed;
01163    inset: 0 auto 0 0;
01164    width: 232px;
01165    background: var(--nav);
01166    color: #d5e4df;
01167    padding: 34px 22px 0;
01168    display: flex;
01169    flex-direction: column;
01170    z-index: 20;
01171  }
01172  .brand {
01173    display: flex;
01174    align-items: center;
01175    gap: 11px;
01176    color: #fff !important;
01177    font-size: 22px;
01178    letter-spacing: -0.6px;
01179    font-weight: 650;
01180    margin-bottom: 48px;
01181  }
01182  .brand-mark {
01183    width: 40px;
01184    height: 44px;
01185    border: 1px solid #ffffff35;
01186    border-radius: 10px;
01187    display: grid;
01188    place-items: center;
01189    background: #ffffff08;
01190  }
01191  .brand-mark svg {
01192    width: 30px;
01193    height: 30px;
01194    stroke: #d4eee0;
01195  }
01196  .brand-sub {
01197    display: block;
01198    font-size: 8px;
01199    letter-spacing: 2.1px;
01200    font-weight: 500;
01201    color: #a1bcb2;
01202    margin-top: 3px;
01203  }
01204  .nav-label,
01205  .eyebrow {
01206    font-size: 10px;
01207    letter-spacing: 1.4px;
01208    font-weight: 650;
01209    text-transform: uppercase;
01210  }
```

源文件范围：app/static/styles.css:213 → app/static/styles.css:276

```
.nav-label {
  color: #8cac9e;
  padding-left: 13px;
  margin-bottom: 13px;
}
nav {
  display: grid;
  gap: 6px;
}
nav a {
  color: #bed1c8;
  display: flex;
  align-items: center;
  padding: 13px;
  gap: 12px;
  border-radius: 9px;
  font-size: 13px;
  transition: 0.16s;
}
nav a:hover {
  background: #ffffff0d;
  color: white;
}
nav a.active {
  background: #e8f2e7;
  color: #174a3e;
  font-weight: 650;
}
.nav-icon {
  display: flex;
  align-items: center;
}
.nav-icon svg {
  width: 18px;
  height: 18px;
}
.nav-count {
  margin-left: auto;
  font-size: 10px;
  min-width: 20px;
  text-align: center;
  padding: 2px 5px;
  border-radius: 5px;
  background: #215d4b;
  color: white;
}
.sidebar-tip {
  border-top: 1px solid #ffffff17;
  margin-top: auto;
  padding: 25px 10px 27px;
}
.sidebar-tip .eyebrow {
  color: #a1c7b6;
  letter-spacing: 0.5px;
  font-size: 11px;
```

源文件范围：app/static/styles.css:277 → app/static/styles.css:340

```
01266  }
01267  .sidebar-tip p {
01268    color: #a2b9ae;
01269    font-size: 11px;
01270    line-height: 1.9;
01271    margin: 10px 0 17px;
01272  }
01273  .demo-button {
01274    color: #d8e8d9;
01275    padding: 0;
01276    border: 0;
01277    background: none;
01278    font-size: 12px;
01279    display: flex;
01280    width: 100%;
01281    justify-content: space-between;
01282    align-items: center;
01283  }
01284  .demo-button:hover {
01285    color: white;
01286  }
01287  .demo-button span {
01288    font-size: 19px;
01289  }
01290  .sidebar-footer {
01291    border-top: 1px solid #ffffff17;
01292    padding: 22px 0;
01293    font-size: 10px;
01294    color: #a6c0b2;
01295    display: flex;
01296    gap: 7px;
01297    align-items: center;
01298  }
01299  .version-label {
01300    margin-left: auto;
01301    font-size: 9px;
01302    color: #77998a;
01303  }
01304  .status-dot {
01305    width: 6px;
01306    height: 6px;
01307    border-radius: 50%;
01308    background: #90c2a2;
01309    display: inline-block;
01310    flex: none;
01311  }
01312  .workspace {
01313    margin-left: 232px;
01314    min-height: 100vh;
01315    display: flex;
01316    flex-direction: column;
01317  }
01318  .topbar {
01319    height: 79px;
01320    border-bottom: 1px solid var(--line);
```

源文件范围：app/static/styles.css:341 → app/static/styles.css:403

```
01321    padding: 0 40px;
01322    display: flex;
01323    align-items: center;
01324    justify-content: space-between;
01325    background: #fafbf8;
01326  }
01327  .breadcrumb {
01328    font-size: 12px;
01329    color: #87928a;
01330    display: flex;
01331    gap: 14px;
01332    align-items: center;
01333  }
01334  .breadcrumb span:first-child {
01335    color: #bcc7bd;
01336  }
01337  .breadcrumb span:last-child {
01338    color: #435d50;
01339  }
01340  .operator {
01341    display: flex;
01342    gap: 9px;
01343    align-items: center;
01344    font-size: 11px;
01345    color: #42594d;
01346  }
01347  .operator-avatar {
01348    display: grid;
01349    place-items: center;
01350    width: 32px;
01351    height: 32px;
01352    border-radius: 50%;
01353    border: 1px solid #d6dfd3;
01354    background: #e7eddf;
01355    color: #687b60;
01356    font-size: 11px;
01357  }
01358  .operator-caption {
01359    font-size: 9px;
01360    color: #8a9689;
01361    display: block;
01362    margin-top: 4px;
01363  }
01364  main {
01365    padding: 35px 40px;
01366    max-width: 1680px;
01367    width: 100%;
01368    margin: 0 auto;
01369    flex: 1;
01370    outline: none !important;
01371  }
01372  .page-footer {
01373    padding: 18px 40px 24px;
01374    display: flex;
01375    justify-content: space-between;
```

源文件范围：app/static/styles.css:404 → app/static/styles.css:466

```
01376    font-size: 10px;
01377    color: #919c91;
01378    gap: 12px;
01379  }
01380  .page-heading {
01381    display: flex;
01382    justify-content: space-between;
01383    align-items: center;
01384    gap: 20px;
01385    margin-bottom: 27px;
01386  }
01387  .page-heading .eyebrow {
01388    display: block;
01389    color: var(--teal);
01390    margin-bottom: 10px;
01391  }
01392  .subtitle {
01393    color: var(--muted);
01394    font-size: 12px;
01395    margin: 0;
01396    line-height: 1.8;
01397  }
01398  .actions {
01399    display: flex;
01400    align-items: center;
01401    gap: 9px;
01402    flex-wrap: wrap;
01403  }
01404  .button {
01405    display: inline-flex;
01406    align-items: center;
01407    justify-content: center;
01408    gap: 7px;
01409    min-height: 38px;
01410    padding: 9px 15px;
01411    border: 1px solid #d8e1d7;
01412    background: white;
01413    color: #39574b;
01414    font-size: 12px;
01415    font-weight: 550;
01416    white-space: nowrap;
01417    border-radius: 8px;
01418  }
01419  .button:hover {
01420    background: #f5f9f3;
01421    border-color: #aec7b3;
01422  }
01423  .button.primary {
01424    background: var(--teal);
01425    color: white;
01426    border-color: var(--teal);
01427    box-shadow: 0 3px 5px #165e4710;
01428  }
01429  .button.primary:hover {
01430    background: #125a4c;
```

源文件范围：app/static/styles.css:467 → app/static/styles.css:531

```
}
.button.subtle {
  background: #f4f7f1;
}
.button.small {
  padding: 6px 10px;
  min-height: 30px;
  font-size: 11px;
}
.button.danger {
  color: var(--red);
  border-color: #eacac4;
  background: white;
}
.button.danger:hover {
  background: var(--red-soft);
}
.text-button {
  background: none;
  border: 0;
  color: var(--teal);
  padding: 3px;
  font-size: 12px;
}
.icon-button {
  width: 32px;
  height: 32px;
  border: 1px solid var(--line);
  background: #fff;
  color: #5f7066;
  display: grid;
  place-items: center;
  font-size: 23px;
  flex: none;
}
.icon-button:hover {
  background: #f0f4ef;
}
.stats {
  display: grid;
  grid-template-columns: repeat(4, minmax(0, 1fr));
  gap: 16px;
  margin-bottom: 25px;
}
.stat-card {
  background: white;
  border: 1px solid var(--line);
  border-radius: var(--radius);
  padding: 19px 20px 17px;
  position: relative;
  overflow: hidden;
}
.stat-label {
  color: #6c7e70;
  font-size: 11px;
```

源文件范围：app/static/styles.css:532 → app/static/styles.css:596

```
  margin-bottom: 18px;
  display: flex;
  justify-content: space-between;
  align-items: center;
}
.stat-label svg {
  color: #8aa68f;
  width: 18px;
  height: 18px;
}
.stat-number {
  font-size: 32px;
  font-weight: 600;
  letter-spacing: -1px;
  line-height: 1.1;
  margin-bottom: 8px;
  font-variant-numeric: tabular-nums;
}
.stat-hint {
  font-size: 10px;
  color: #879485;
}
.stat-card.accent {
  background: #eef4e9;
  border-color: #d7e3cf;
}
.stat-card.accent .stat-number {
  color: var(--teal);
}
.dashboard-grid {
  display: grid;
  grid-template-columns: minmax(0, 1.55fr) minmax(270px, 1fr);
  gap: 22px;
}
.panel {
  background: white;
  border: 1px solid var(--line);
  border-radius: var(--radius);
  overflow: hidden;
  margin-bottom: 22px;
}
.panel-header {
  display: flex;
  justify-content: space-between;
  align-items: center;
  padding: 20px 22px;
  border-bottom: 1px solid #edf0e9;
  gap: 12px;
}
.panel-header h2 {
  font-size: 14px;
  margin: 0;
}
.panel-header p {
  margin: 5px 0 0;
```

源文件范围：app/static/styles.css:597 → app/static/styles.css:659

```
01541    font-size: 11px;
01542    color: var(--muted);
01543  }
01544  .panel-body {
01545    padding: 22px;
01546  }
01547  .welcome-panel {
01548    background: #f0f5e9;
01549    border: 1px solid #d8e4ce;
01550    border-radius: var(--radius);
01551    padding: 27px 26px;
01552    position: relative;
01553    margin-bottom: 22px;
01554    overflow: hidden;
01555  }
01556  .welcome-panel:after {
01557    content: "";
01558    width: 120px;
01559    height: 120px;
01560    border: 24px solid #dce7cc;
01561    border-radius: 50%;
01562    position: absolute;
01563    right: -30px;
01564    top: -30px;
01565    opacity: 0.5;
01566    pointer-events: none;
01567  }
01568  .welcome-panel .eyebrow {
01569    color: #6b8260;
01570    font-size: 9px;
01571  }
01572  .welcome-panel h2 {
01573    font-size: 20px;
01574    margin-top: 13px;
01575    margin-bottom: 12px;
01576    line-height: 1.6;
01577    max-width: 260px;
01578  }
01579  .welcome-panel p {
01580    font-size: 11px;
01581    color: #708365;
01582    max-width: 270px;
01583    margin-bottom: 19px;
01584    position: relative;
01585    z-index: 1;
01586  }
01587  .welcome-panel .button {
01588    position: relative;
01589    z-index: 1;
01590  }
01591  .workflow {
01592    padding: 0;
01593    list-style: none;
01594    margin: 0;
01595    counter-reset: step;
```

源文件范围：app/static/styles.css:660 → app/static/styles.css:723

```
}
.workflow li {
  display: flex;
  gap: 12px;
  padding-bottom: 19px;
  position: relative;
}
.workflow li:last-child {
  padding: 0;
}
.workflow li:not(:last-child):after {
  content: "";
  position: absolute;
  top: 27px;
  left: 13px;
  bottom: 3px;
  border-left: 1px dashed #cfdccd;
}
.step-number {
  width: 27px;
  height: 27px;
  border-radius: 8px;
  background: #f0f5ec;
  color: #5b8063;
  font-size: 11px;
  display: grid;
  place-items: center;
  flex: none;
}
.workflow strong {
  font-weight: 550;
  font-size: 12px;
  display: block;
  margin: 2px 0 5px;
}
.workflow p {
  font-size: 10px;
  margin: 0;
  color: #8a9686;
  line-height: 1.7;
}
.table-scroll {
  overflow-x: auto;
}
table {
  width: 100%;
  border-collapse: collapse;
  text-align: left;
}
th {
  background: #f8faf6;
  color: #7b8977;
  font-size: 10px;
  font-weight: 550;
  letter-spacing: 0.2px;
```

源文件范围：app/static/styles.css:2098 → app/static/styles.css:2153

```
    .report-links {
      flex-wrap: wrap;
    }
    .comparison-heading .actions {
      width: 100%;
    }
    th {
      padding: 11px 16px;
    }
    td {
      padding: 14px 16px;
    }
    .table-scroll {
      border-radius: 0 0 14px 14px;
    }
    .toasts {
      left: 18px;
      right: 18px;
      bottom: 18px;
      max-width: none;
    }
    .toast {
      font-size: 11px;
    }
    .form-grid {
      grid-template-columns: 1fr;
    }
    .span-2 {
      grid-column: auto;
    }
    .dialog-header {
      padding: 20px;
    }
    .dialog-header h2 {
      font-size: 18px;
    }
    #modal-content {
      padding: 20px;
    }
    .sticky-form-actions {
      bottom: -20px;
    }
    .audit-detail {
      min-width: 220px;
    }
    .version-card-grid {
      grid-template-columns: 1fr;
    }
    .welcome-panel {
      margin-bottom: 20px;
    }
  }
  @media (prefers-reduced-motion: reduce) {
    *,
    *:before,
```

源文件范围：app/static/styles.css:2154 → app/static/index.html:11

```
    *:after {
      animation: none !important;
      transition: none !important;
      scroll-behavior: auto !important;
    }
  }
  @media print {
    .sidebar,
    .topbar,
    .page-footer,
    .actions,
    .filter-bar,
    .review-controls,
    dialog,
    .toasts {
      display: none !important;
    }
    .workspace {
      margin: 0;
    }
    main {
      padding: 0;
    }
    .change-card,
    .panel {
      break-inside: avoid;
      box-shadow: none;
    }
    .diff-grid {
      grid-template-columns: 1fr 1fr;
    }
    .diff-text {
      font-size: 10pt;
    }
    .diff-column:first-child {
      border-right: 1px solid #ddd;
    }
    .stats {
      grid-template-columns: repeat(4, 1fr);
    }
    body {
      background: white;
    }
  }
  <!doctype html>
  <html lang="zh-CN">
    <head>
      <meta charset="utf-8" />
      <meta name="viewport" content="width=device-width,initial-scale=1" />
      <meta name="color-scheme" content="light" />
      <meta name="theme-color" content="#123d3b" />
      <title>MedLabel Review · 药品说明书审核</title>
      <link
        rel="icon"
        href="data:image/svg+xml,%3Csvg xmlns='http://www.w3.org/2000/svg' viewBox='
```

源文件范围：app/static/index.html:11 → app/static/index.html:63

```
02971  0 0 48 48'%3E%3Crect width='48' height='48' rx='13' fill='%23174441'/%3E%3Cpath d=
02972  'M15 12h18v24H15z' fill='none' stroke='white' stroke-width='2'/%3E%3Cpath d='m19 2
02973  5 4 4 7-9M20 17h8' fill='none' stroke='%239ce1c7' stroke-width='2'/%3E%3C/svg%3E"
02974      />
02975      <link rel="stylesheet" href="/static/styles.css" />
02976      <script src="/static/app.js" defer></script>
02977    </head>
02978    <body>
02979      <a class="skip-link" href="#main">跳至主要内容</a>
02980      <aside class="sidebar" aria-label="主导航">
02981        <a class="brand" href="#dashboard" aria-label="MedLabel Review 工作台">
02982          <span class="brand-mark" aria-hidden="true"
02983            ><svg viewBox="0 0 32 32">
02984              <path d="M8 5h12l4 4v18H8zM20 5v5h4M12 13h8m-8 5 3 3 6-6" /></svg
02985          ></span>
02986          <span>MedLabel<span class="brand-sub">REVIEW WORKSPACE</span></span>
02987        </a>
02988        <div class="nav-label">说明书管理</div>
02989        <nav id="navigation">
02990          <a href="#dashboard" data-nav="dashboard"
02991            ><span class="nav-icon" data-icon="grid"></span>工作台</a
02992          >
02993          <a href="#drugs" data-nav="drugs"
02994            ><span class="nav-icon" data-icon="archive"></span>药品与版本</a
02995          >
02996          <a href="#comparisons" data-nav="comparisons"
02997            ><span class="nav-icon" data-icon="compare"></span>比对与审核<span
02998              id="nav-pending"
02999              class="nav-count"
03000              hidden
03001            ></span
03002          ></a>
03003          <a href="#audit" data-nav="audit"
03004            ><span class="nav-icon" data-icon="clock"></span>操作记录</a
03005          >
03006        </nav>
03007        <div class="sidebar-tip">
03008          <span class="eyebrow">让每一次变更都有依据</span>
03009          <p>从原文定位到人工复核，<br />保留完整的审核过程。</p>
03010          <button type="button" class="demo-button" data-action="demo">
03011            体验模拟示例 <span aria-hidden="true">↗</span>
03012          </button>
03013        </div>
03014        <div class="sidebar-footer">
03015          <span class="status-dot"></span> 本地工作空间<span class="version-label"
03016            >V1.0</span
03017          >
03018        </div>
03019      </aside>
03020      <div class="workspace">
03021        <header class="topbar">
03022          <div class="breadcrumb">
03023            药品说明书 <span>/</span> <span id="breadcrumb-current">工作台</span>
03024          </div>
03025          <div class="operator">
```

源文件范围：app/static/index.html:64 → app/static/app.js:4

```
            <span class="operator-avatar" aria-hidden="true">审</span
            ><span
              >本地单人审核<span class="operator-caption"
                >资料保存在本机</span
              ></span
            >
          </div>
        </header>
        <main id="main" tabindex="-1" aria-live="polite">
          <div class="loading-state">
            <span class="spinner"></span>正在加载工作空间…
          </div>
        </main>
        <footer class="page-footer">
          <span>药品说明书版本比对与变更审核系统</span
          ><span>辅助资料核查 · 变更需人工复核</span>
        </footer>
      </div>
      <dialog id="modal" aria-labelledby="modal-title">
        <div class="dialog-shell">
          <header class="dialog-header">
            <div>
              <span id="modal-eyebrow" class="eyebrow">MEDLABEL REVIEW</span>
              <h2 id="modal-title"></h2>
            </div>
            <button
              type="button"
              class="icon-button"
              id="modal-close"
              aria-label="关闭对话框"
            >
              ×
            </button>
          </header>
          <div id="modal-content"></div>
        </div>
      </dialog>
      <div
        id="toasts"
        class="toasts"
        role="status"
        aria-live="polite"
        aria-atomic="true"
      ></div>
      <noscript
        ><div class="noscript">
          本应用需要启用 JavaScript 才能管理药品说明书。
        </div></noscript
      >
    </body>
  </html>
  /* MedLabel Review — local document review workspace. No remote assets or services
  . */
  "use strict";
  const $ = (selector, root = document) => root.querySelector(selector);
```

源文件范围：app/static/app.js:5 → app/static/app.js:56

```
const $$ = (selector, root = document) =>
  Array.from(root.querySelectorAll(selector));
const main = $("#main");
const modal = $("#modal");
const state = {
  route: "",
  generation: 0,
  comparison: null,
  reviewStatus: "all",
  priority: "all",
  drafts: new Map(),
  drugs: [],
  comparisons: [],
  audit: [],
  modalVersion: null,
};
const glyphs = {
  grid: '<rect x="3" y="3" width="7" height="7" rx="1.5"/><rect x="14" y="3" width
="7" height="7" rx="1.5"/><rect x="3" y="14" width="7" height="7" rx="1.5"/><rect
x="14" y="14" width="7" height="7" rx="1.5"/>',
  archive: '<path d="M4 8h16v12H4zM3 4h18v4H3zM9 12h6"/>',
  compare: '<path d="M3 7h16m-4-4 4 4-4 4M21 17H5m4-4-4 4 4 4"/>',
  clock: '<circle cx="12" cy="12" r="9"/><path d="M12 7v5l3 2"/>',
  plus: '<path d="M12 5v14M5 12h14"/>',
  file: '<path d="M6 3h8l4 4v14H6zM14 3v5h4M9 12h6m-6 4h6"/>',
  upload: '<path d="M12 16V3m-5 5 5-5 5 5M4 15v6h16v-6"/>',
  download: '<path d="M12 3v13m-5-5 5 5 5-5M4 17v4h16v-4"/>',
  search: '<circle cx="10.5" cy="10.5" r="6.5"/><path d="m16 16 5 5"/>',
  check: '<path d="m5 12 4 4L19 6"/>',
  shield: '<path d="m12 3 8 3v6c0 5-8 9-8 9s-8-4-8-9V6zM8 12l3 3 5-6"/>',
  arrow: '<path d="M4 12h16m-6-6 6 6-6 6"/>',
  back: '<path d="M20 12H4m6-6-6 6 6 6"/>',
  warning: '<path d="M12 3 2 21h20zM12 9v5m0 3v1"/>',
  edit: '<path d="m14 5 5 5M4 20l1-6L16 3l5 5-11 11z"/>',
  filter: '<path d="M3 5h18l-7 8v7l-4-2v-5z"/>',
  spark:
    '<path d="m12 3 2.5 6.5L21 12l-6.5 2.5L12 21l-2.5-6.5L3 12l6.5-2.5z"/>',
};
function icon(name) {
  return `<svg viewBox="0 0 24 24" aria-hidden="true">${glyphs[name] || glyphs.fil
e}</svg>`;
}
function esc(value) {
  return String(value ?? "").replace(
    /[&<>"']/g,
    (character) =>
      ({ "&": "&amp;", "<": "&lt;", ">": "&gt;", '"': "&quot;", "'": "&#39;" })[
        character
      ],
  );
}
function id(value) {
  return Number.isInteger(Number(value)) && Number(value) > 0
    ? String(Number(value))
    : "0";
```

源文件范围：app/static/app.js:57 → app/static/app.js:103

```
}
function fmtDate(value, includeTime = false) {
  if (!value) return "未记录";
  const date = new Date(value);
  if (Number.isNaN(date.getTime())) return String(value);
  return new Intl.DateTimeFormat("zh-CN", {
    year: "numeric",
    month: "2-digit",
    day: "2-digit",
    ...(includeTime ? { hour: "2-digit", minute: "2-digit" } : {}),
  }).format(date);
}
function statusTag(status) {
  const options = {
    pending: ["待复核", "amber"],
    confirmed: ["已确认", "teal"],
    ignored: ["已忽略", "gray"],
    completed: ["已完成", "teal"],
    complete: ["已完成", "teal"],
    reviewed: ["已完成", "teal"],
  };
  const [label, color] = options[status] || ["待复核", "amber"];
  return `<span class="tag ${color}"><span class="badge-pending">${label}</span></
span>`;
}
function kindTag(kind) {
  return `<span class="tag ${kind === "added" ? "green" : kind === "removed" ? "re
d" : "gray"}">${{ added: "新增", removed: "删除", modified: "修改" }[kind] || esc(
kind)}</span>`;
}
function pageHeading(eyebrow, title, subtitle, actions = "", extra = "") {
  return `<div class="page-heading ${extra}"><div><span class="eyebrow">${esc(eyeb
row)}</span><h1>${esc(title)}</h1><p class="subtitle">${esc(subtitle)}</p></div>${
actions ? `<div class="actions">${actions}</div>` : ""}</div>`;
}
function empty(title, description, action = "", compact = false) {
  return `<div class="empty-state ${compact ? "compact" : ""}">${icon("file")}<h3>
${esc(title)}</h3><p>${esc(description)}</p>${action}</div>`;
}
function button(label, action, data = "", primary = false, glyph = "") {
  return `<button type="button" class="button${primary ? " primary" : ""}" data-ac
tion="${action}" ${data}>${glyph ? icon(glyph) : ""}${esc(label)}</button>`;
}
function warnings(items) {
  if (!items?.length) return "";
  return `<div class="notice">${icon("warning")}<p>${items.map((item) => esc(item)
).join("<br>")}</p></div>`;
}
async function api(path, options = {}) {
  const headers = new Headers(options.headers || {});
  if (options.method && options.method !== "GET")
    headers.set("X-Requested-With", "MedLabelReview");
  if (options.body && !(options.body instanceof FormData)) {
    headers.set("Content-Type", "application/json");
    options.body = JSON.stringify(options.body);
```

源文件范围：app/static/app.js:104 → app/static/app.js:155

```
  }
  let response;
  try {
    response = await fetch(path, {
      ...options,
      headers,
      credentials: "same-origin",
    });
  } catch (_) {
    throw new Error("无法连接本地服务，请确认应用已启动后重试。");
  }
  let result;
  try {
    result = await response.json();
  } catch (_) {
    throw new Error("服务返回了无法读取的内容，请稍后重试。");
  }
  if (!response.ok) {
    let detail =
      result.detail || result.message || `请求失败（${response.status}）`;
    if (Array.isArray(detail))
      detail = detail
        .map(
          (item) =>
            `${(item.loc || []).filter((p) => p !== "body").join(" / ")}：${item.m
sg}`,
        )
        .join("；");
    const error = new Error(
      typeof detail === "string" ? detail : JSON.stringify(detail),
    );
    error.status = response.status;
    throw error;
  }
  return result;
}
function toast(message, error = false) {
  const element = document.createElement("div");
  element.className = `toast${error ? " error" : ""}`;
  element.innerHTML = `${icon(error ? "warning" : "check")}<span>${esc(message)}</
span>`;
  $("#toasts").append(element);
  setTimeout(() => element.remove(), error ? 7500 : 4200);
}
function renderError(error) {
  main.innerHTML = `<div class="error-state"><h2>暂时无法加载</h2><p>${esc(error.m
essage)}</p>${button("重新加载", "refresh", "", "", "arrow")}</div>`;
}
function modalError(message) {
  let container = $(".dialog-error", modal);
  if (!container) {
    container = document.createElement("div");
    container.className = "dialog-error";
    container.setAttribute("role", "alert");
    $("#modal-content").prepend(container);
```

源文件范围：app/static/app.js:156 → app/static/app.js:211

```
  }
  container.textContent = message;
  container.scrollIntoView({ block: "nearest" });
}
function showModal(title, content, wide = false) {
  $("#modal-title").textContent = title;
  $("#modal-content").innerHTML = content;
  modal.classList.toggle("wide", wide);
  if (!modal.open) modal.showModal();
  // Set initial focus synchronously; a delayed callback can steal focus after
  // the user has already moved to another field and misdirect their input.
  const input = $('input:not([type="hidden"]), textarea, select', modal);
  (input || $("#modal-close")).focus();
}
function closeModal() {
  modal.close();
  state.modalVersion = null;
}
$("#modal-close").addEventListener("click", closeModal);
modal.addEventListener("click", (event) => {
  if (event.target === modal) {
    const bounds = modal.getBoundingClientRect();
    if (
      event.clientX < bounds.left ||
      event.clientX > bounds.right ||
      event.clientY < bounds.top ||
      event.clientY > bounds.bottom
    )
      closeModal();
  }
});
$$("[data-icon]").forEach((element) => {
  element.innerHTML = icon(element.dataset.icon);
});
async function navigate() {
  const route = location.hash.slice(1) || "dashboard";
  const generation = ++state.generation;
  const previous = state.route;
  state.route = route;
  if (previous !== route) {
    state.reviewStatus = "all";
    state.priority = "all";
  }
  const parts = route.split("/");
  const section = ["dashboard", "drugs", "comparisons", "audit"].includes(
    parts[0],
  )
    ? parts[0]
    : "dashboard";
  $$("[data-nav]").forEach((element) => {
    const selected = element.dataset.nav === section;
    element.classList.toggle("active", selected);
    if (selected) element.setAttribute("aria-current", "page");
    else element.removeAttribute("aria-current");
  });
```

源文件范围：app/static/app.js:212 → app/static/app.js:267

```
    $("#breadcrumb-current").textContent = {
      dashboard: "工作台",
      drugs: parts[1] ? "药品档案" : "药品与版本",
      comparisons: parts[1] ? "变更审核" : "比对与审核",
      audit: "操作记录",
    }[section];
    main.innerHTML =
      '<div class="loading-state"><span class="spinner"></span>正在加载…</div>';
    try {
      let html;
      if (section === "drugs" && parts[1])
        html = await drugPage(Number(parts[1]));
      else if (section === "drugs") html = await drugsPage();
      else if (section === "comparisons" && parts[1])
        html = await comparisonPage(Number(parts[1]));
      else if (section === "comparisons") html = await comparisonsPage();
      else if (section === "audit") html = await auditPage();
      else html = await dashboardPage();
      if (generation !== state.generation) return;
      main.innerHTML = html;
      document.title = `${$("#breadcrumb-current").textContent} · MedLabel Review`;
      if (previous !== route) window.scrollTo(0, 0);
    } catch (error) {
      if (generation === state.generation) renderError(error);
    }
  }
  window.addEventListener("hashchange", navigate);
  function auditLabel(action) {
    const mapping = {
      "drug.create": "建立药品档案",
      "version.import": "导入说明书版本",
      "version.create": "导入说明书版本",
      "version.correct": "校正说明书章节",
      "version.sections.update": "校正说明书章节",
      "comparison.create": "创建版本比对",
      "change.review": "更新审核结论",
      "demo.seed": "载入模拟示例",
      create_drug: "建立药品档案",
      import_version: "导入说明书版本",
      correct_sections: "校正说明书章节",
      create_comparison: "创建版本比对",
      review_change: "更新审核结论",
    };
    return mapping[action] || action;
  }
  function detailText(detail) {
    let value = detail;
    if (typeof value === "string") {
      try {
        value = JSON.parse(value);
      } catch (_) {
        return value;
      }
    }
    if (!value || typeof value !== "object") return String(value ?? "");
```

源文件范围：app/static/app.js:268 → app/static/app.js:317

```
  if (Array.isArray(value.before) && Array.isArray(value.after)) {
    return `章节校正：${value.before.length} 个 → ${value.after.length} 个章节`;
  }
  if (value.before?.status && value.after?.status) {
    const labels = {
      pending: "待复核",
      confirmed: "已确认",
      ignored: "已忽略",
    };
    return `${labels[value.before.status] || value.before.status} → ${labels[value
.after.status] || value.after.status}${value.after.note ? `；复核意见：${value.aft
er.note}` : ""}`;
  }
  const labels = {
    name: "药品",
    label: "版本",
    filename: "文件",
    drug_id: "药品档案",
    old_version_id: "旧版编号",
    new_version_id: "新版编号",
    changes: "变更数",
    comparison_id: "比对编号",
    note: "复核意见",
    status: "状态",
  };
  return Object.entries(value)
    .filter(([key]) => key !== "sha256")
    .map(
      ([key, item]) =>
        `${labels[key] || key}：${typeof item === "object" ? JSON.stringify(item)
: item}`,
    )
    .join(" · ");
}
function activityList(items, limit = 5) {
  if (!items?.length)
    return empty(
      "尚无操作记录",
      "导入说明书后，操作记录会显示在这里。",
      "",
      true,
    );
  return `<ul class="activity">${items
    .slice(0, limit)
    .map(
      (item) =>
        `<li><span class="activity-dot"></span><div class="activity-content"><stro
ng>${esc(auditLabel(item.action))}</strong><p>${esc(detailText(item.detail))}</p><
/div><time>${esc(fmtDate(item.created_at))}</time></li>`,
    )
    .join("")}</ul>`;
}
async function dashboardPage() {
  const [data, comparisons] = await Promise.all([
    api("/api/dashboard"),
```

源文件范围：app/static/app.js:318 → app/static/app.js:351

```
      api("/api/comparisons"),
    ]);
    const count = Number(data.pending_changes || 0);
    $("#nav-pending").textContent = count;
    $("#nav-pending").hidden = !count;
    const stats = [
      ["药品档案", data.drugs, "已建立的药品资料", "archive"],
      ["说明书版本", data.versions, "保留来源与原始文件", "file"],
      ["比对任务", data.comparisons, "可追溯的版本变更", "compare"],
      ["待复核变更", count, "等待人工确认与处理", "shield"],
    ];
    return (
      pageHeading(
        "YOUR REVIEW WORKSPACE",
        "每一次变更，都清晰可见",
        "管理说明书版本，让变更识别、复核与归档有序进行。",
        button("新建药品", "new-drug", "", true, "plus"),
      ) +
      `<div class="stats">${stats.map(([label, value, hint, glyph], index) => `<div
class="stat-card ${index === 3 ? "accent" : ""}"><div class="stat-label">${label}$
{icon(glyph)}</div><div class="stat-number">${esc(value ?? 0)}</div><div class="st
at-hint">${hint}</div></div>`).join("")}</div>` +
      `<div class="dashboard-grid"><div><section class="panel"><header class="panel-
header"><div><h2>最近的比对任务</h2><p>从差异发现到审核完成</p></div><a class="tex
t-button" href="#comparisons">查看全部 →</a></header>${
        comparisons.length
          ? comparisons
              .slice(0, 5)
              .map(
                (item) =>
                  `<a class="list-row row-link" href="#comparisons/${id(item.id)}"><
span class="list-icon">${icon("compare")}</span><div class="list-details"><strong>
${esc(item.drug_name || "说明书版本比对")}</strong><p>${esc(item.old_label)} → ${e
sc(item.new_label)} · ${esc(item.total_changes ?? 0)} 处变更</p></div>${statusTag(
Number(item.pending_changes) ? "pending" : "completed")}<span class="list-arrow">›
</span></a>`,
              )
              .join("")
          : empty(
              "开启第一份说明书审核",
              "建立药品档案，导入两个版本，即可生成章节差异。",
              button("体验模拟示例", "demo", "", "", "spark"),
            )
      }</section><section class="panel"><header class="panel-header"><h2>最近操作</h
2><a href="#audit" class="text-button">完整记录 →</a></header><div class="panel-bo
dy">${activityList(data.recent_activity)}</div></section></div><aside class="dashb
oard-aside"><div class="welcome-panel"><span class="eyebrow">A CLEARER REVIEW PROC
ESS</span><h2>关注重要变化，<br>保留判断依据。</h2><p>对齐说明书章节，定位数字、单
位和重点条款变化，逐条记录人工复核意见。</p><a class="button" href="#drugs">进入药
品资料库 ${icon("arrow")}</a></div><section class="panel"><header class="panel-hea
der"><h2>四步完成审核</h2></header><div class="panel-body"><ol class="workflow"><l
i><span class="step-number">01</span><div><strong>建立档案与导入版本</strong><p>支
持纯文字 PDF、DOCX、TXT 和粘贴文本</p></div></li><li><span class="step-number">02<
/span><div><strong>检查章节，生成比对</strong><p>必要时先校正识别结果，再固定比对
依据</p></div></li><li><span class="step-number">03</span><div><strong>逐条人工复
```

源文件范围：app/static/app.js:351 → app/static/app.js:391

```
核</strong><p>确认或忽略变更，保存审核意见</p></div></li><li><span class="step-num
ber">04</span><div><strong>导出报告，留存记录</strong><p>HTML、CSV、JSON 三种格式<
/p></div></li></ol></div></section></aside></div>`
  );
}
async function drugsPage() {
  state.drugs = await api("/api/drugs");
  return (
    pageHeading(
      "DOCUMENT LIBRARY",
      "药品与版本",
      "以药品为单位归集说明书，保留每一版的来源与原文。",
      button("新建药品", "new-drug", "", true, "plus"),
    ) +
    `<section class="panel"><div class="toolbar"><div class="search-wrap">${icon("
search")}<input type="search" id="drug-search" aria-label="搜索药品名称、厂家或批
准文号" placeholder="搜索药品、厂家或批准文号"></div><span class="count-label" id=
"drug-count">${state.drugs.length} 个药品档案</span></div><div id="drug-list">${dr
ugTable(state.drugs)}</div></section>`
  );
}
function drugTable(items) {
  if (!items.length)
    return empty(
      "没有找到药品档案",
      "新建档案后，可以导入说明书并开始版本比对。",
      button("新建药品", "new-drug", "", true, "plus"),
    );
  return `<div class="table-scroll"><table><thead><tr><th>药品名称 / 规格</th><th>
生产厂家</th><th>批准文号</th><th>版本</th><th>建立日期</th><th></th></tr></thead>
<tbody>${items.map((item) => `<tr><td><a class="cell-title" href="#drugs/${id(item
.id)}">${esc(item.name)}</a><span class="cell-sub">${esc(item.specification || "未
填写规格")}</span></td><td>${esc(item.manufacturer || "—")}</td><td class="mono">$
{esc(item.approval_number || "—")}</td><td><span class="tag teal">${esc(item.versi
on_count ?? 0)} 版</span></td><td class="nowrap muted">${esc(fmtDate(item.created_
at))}</td><td class="text-right"><a class="button small" href="#drugs/${id(item.id
)}">查看档案 →</a></td></tr>`).join("")}</tbody></table></div>`;
}
async function drugPage(drugId) {
  const [drug, versions] = await Promise.all([
    api(`/api/drugs/${id(drugId)}`),
    api(`/api/drugs/${id(drugId)}/versions`),
  ]);
  const fields = [
    ["生产厂家", drug.manufacturer],
    ["药品规格", drug.specification],
    ["批准文号", drug.approval_number],
    ["建立日期", fmtDate(drug.created_at)],
  ];
  return (
    `<a class="back-link" href="#drugs">${icon("back")} 返回药品资料库</a>` +
    pageHeading(
      "DRUG DOSSIER",
      drug.name,
      "保存来源，核对章节，再开始版本审核。",
```

源文件范围：app/static/app.js:392 → app/static/app.js:419

```
        button(
          "导入版本",
          "import-version",
          `data-id="${id(drugId)}"`,
          true,
          "upload",
        ) +
          button(
            "创建比对",
            "new-comparison",
            `data-id="${id(drugId)}" ${versions.length < 2 ? 'disabled title="请先导入至少两个版本"' : ""}`,
            "",
            "compare",
          ),
      ) +
      `<section class="panel"><div class="panel-body"><div class="metadata-grid">${fields.map(([label, value]) => `<div><span class="metadata-label">${label}</span><span class="metadata-value">${esc(value || "未填写")}</span></div>`).join("")}</div>${drug.notes ? `<div class="metadata-note">${esc(drug.notes)}</div>` : ""}</div></section><div class="page-heading"><div><h2>说明书版本</h2><p class="subtitle">共 ${versions.length} 个版本 · 导入后请检查章节识别结果</p></div></div>${versions.length ? `<div class="version-card-grid">${versions.map((version) => `<article class="version-card"><div class="version-card-head"><div><h3>${esc(version.label)}</h3><span class="cell-sub">版本日期 ${esc(fmtDate(version.version_date))}</span></div><span class="list-icon">${icon("file")}</span></div><p>文件：${esc(version.filename || "粘贴文本")}</p><p>来源：${esc(version.source || "未填写")}</p><p>识别章节：${esc(version.section_count ?? 0)} 个</p><p class="hash" title="原始内容 SHA-256">SHA-256 ${esc(String(version.sha256 || "").slice(0, 20))}…</p><div class="actions">${button("查看与校正", "view-version", `data-id="${id(version.id)}"`, "", "edit")}<a class="button small" href="/api/versions/${id(version.id)}/file">原始文件 ${icon("download")}</a></div></article>`).join("")}</div>` : `<section class="panel">${empty("还没有说明书版本", "先导入旧版和新版说明书，随后可查看章节并创建比对。", button("导入第一个版本", "import-version", `data-id="${id(drugId)}"`, true, "upload"))}</section>`}`
    );
  }
  async function comparisonsPage() {
    state.comparisons = await api("/api/comparisons");
    return (
      pageHeading(
        "REVIEW QUEUE",
        "比对与审核",
        "逐项核查版本差异，保留复核结论与变更依据。",
        `<a class="button primary" href="#drugs">${icon("plus")} 从药品档案创建比对</a>`,
      ) +
      `<section class="panel"><div class="toolbar"><div class="search-wrap">${icon("search")}<input type="search" id="comparison-search" aria-label="搜索比对任务" placeholder="搜索药品名称或版本"></div><select class="filter-select" id="comparison-status" aria-label="任务状态"><option value="all">全部任务</option><option value="pending">有待复核变更</option><option value="completed">已完成复核</option></select><span class="count-label" id="comparison-count">${state.comparisons.length} 个比对任务</span></div><div id="comparison-list">${comparisonTable(state.comparisons)}</div></section>`
```

源文件范围：app/static/app.js:420 → app/static/app.js:458

```
  );
}
function comparisonTable(items) {
  if (!items.length)
    return empty(
      "暂无匹配的比对任务",
      "在药品档案中选择两个版本，创建一份新的比对任务。",
      '<a href="#drugs" class="button primary">前往药品档案 →</a>',
    );
  return `<div class="table-scroll"><table><thead><tr><th>药品 / 版本</th><th>变更
数量</th><th>待复核</th><th>任务状态</th><th>创建时间</th><th></th></tr></thead><t
body>${items.map((item) => `<tr><td><a href="#comparisons/${id(item.id)}" class="c
ell-title">${esc(item.drug_name)}</a><span class="cell-sub">${esc(item.old_label)}
 → ${esc(item.new_label)}</span></td><td>${esc(item.total_changes ?? 0)} 项</td><t
d>${esc(item.pending_changes ?? 0)} 项</td><td>${statusTag(Number(item.pending_cha
nges) ? "pending" : "completed")}</td><td class="nowrap muted">${esc(fmtDate(item.
created_at))}</td><td><a class="button small" href="#comparisons/${id(item.id)}">
打开审核 →</a></td></tr>`).join("")}</tbody></table></div>`;
}
async function comparisonPage(comparisonId) {
  const result = await api(`/api/comparisons/${id(comparisonId)}`);
  if (state.route !== `comparisons/${id(comparisonId)}`) return "";
  state.comparison = result;
  return comparisonMarkup();
}
function comparisonMarkup() {
  const data = state.comparison;
  const changes = data.changes || [];
  const pending = changes.filter(
    (change) => change.status === "pending",
  ).length;
  const oldVersion = data.old_version || {};
  const newVersion = data.new_version || {};
  const progress = changes.length
    ? Math.round(((changes.length - pending) / changes.length) * 100)
    : 100;
  return (
    `<a class="back-link" href="#comparisons">${icon("back")} 返回审核任务</a>` +
    pageHeading(
      "VERSION COMPARISON",
      data.drug_name || "说明书变更审核",
      `${changes.length} 处章节变更 · ${data.unchanged_sections || 0} 个章节内容一
致 · 逐条核对后记录结论`,
      `<div class="report-links"><a class="button" href="/api/comparisons/${id(dat
a.id)}/export?format=html">${icon("download")} HTML 报告</a><a class="button" href
="/api/comparisons/${id(data.id)}/export?format=csv">CSV</a><a class="button" href
="/api/comparisons/${id(data.id)}/export?format=json">JSON</a></div>`,
      "comparison-heading",
    ) +
    warnings(data.warnings) +
    `<div class="comparison-summary"><div class="version-pair"><div class="version
-pill"><span class="eyebrow">旧版说明书</span><strong>${esc(oldVersion.label || da
ta.old_label)}</strong><small>${esc(fmtDate(oldVersion.version_date))}</small></di
v>${icon("arrow")}<div class="version-pill"><span class="eyebrow">新版说明书</span
><strong>${esc(newVersion.label || data.new_label)}</strong><small>${esc(fmtDate(n
```

源文件范围：app/static/app.js:458 → app/static/app.js:494

```
ewVersion.version_date))}</small></div></div><div class="review-progress"><div cla
ss="review-progress-label"><span>审核完成度</span><strong>${changes.length - pendi
ng} / ${changes.length}</strong></div><div class="progress-track" role="progressba
r" aria-label="审核完成度" aria-valuemin="0" aria-valuemax="100" aria-valuenow="${
progress}"><div class="progress-fill" style="width:${progress}%"></div></div></div
></div><div class="filter-bar"><div class="segmented" role="group" aria-label="按
审核状态筛选">${[
      ["all", "全部变更"],
      ["pending", "待复核"],
      ["confirmed", "已确认"],
      ["ignored", "已忽略"],
    ]
      .map(
        ([value, label]) =>
          `<button type="button" data-action="filter-review" data-value="${value}"
 aria-pressed="${state.reviewStatus === value}" class="${state.reviewStatus === va
lue ? "active" : ""}">${label}${value === "pending" ? ` ${pending}` : ""}</button>
`,
      )
      .join(
        "",
      )}</div><select id="priority-filter" class="filter-select" aria-label="按复
核优先级筛选"><option value="all" ${state.priority === "all" ? "selected" : ""}>全
部优先级</option><option value="high" ${state.priority === "high" ? "selected" : "
"}>重点复核</option><option value="normal" ${state.priority === "normal" ? "select
ed" : ""}>常规复核</option></select><span class="small-hint">红色为删除 · 绿色为新
增</span></div><div id="changes-list">${changesMarkup()}</div><p class="comparison
-footer">重点标记用于安排复核顺序，不代表医学风险等级。HTML 报告可通过浏览器打印为
 PDF。</p>`
  );
}
function markedText(text) {
  // Match untrusted plain text first; escaping before matching could split an HTM
L entity.
  const value = String(text ?? "");
  const pattern =
    /\d+(?:\.\d+)?(?:\s*(?:mg|μg|ug|kg|g|ml|mL|L|%|毫克|微克|克|毫升|片|次|日|天))
?/g;
  let result = "";
  let cursor = 0;
  for (const match of value.matchAll(pattern)) {
    result += esc(value.slice(cursor, match.index));
    result += `<span class="numeric">${esc(match[0])}</span>`;
    cursor = match.index + match[0].length;
  }
  return result + esc(value.slice(cursor));
}
function diffText(change, side) {
  const text = side === "old" ? change.old_text : change.new_text;
  if (!text) return '<span class="absent-text">该版本无此章节内容</span>';
  const segments = change.segments;
  if (!Array.isArray(segments) || !segments.length) return esc(text);
  const relevant = segments.filter(
    (segment) =>
      segment.op === "equal" ||
```

源文件范围：app/static/app.js:495 → app/static/app.js:523

```
03686        segment.op === (side === "old" ? "delete" : "insert"),
03687    );
03688    // Only apply inline segments if they reconstruct this exact source; never hide
03689  source text.
03690    if (relevant.map((segment) => segment.text || "").join("") !== text)
03691      return esc(text);
03692    return relevant
03693      .map((segment) =>
03694        segment.op === "equal"
03695          ? esc(segment.text)
03696          : `<${side === "old" ? "del" : "ins"}>${markedText(segment.text)}</${side
03697  === "old" ? "del" : "ins"}>`,
03698      )
03699      .join("");
03700  }
03701  function changesMarkup() {
03702    const changes = state.comparison.changes || [];
03703    const filtered = changes.filter(
03704      (change) =>
03705        (state.reviewStatus === "all" || change.status === state.reviewStatus) &&
03706        (state.priority === "all" || change.priority === state.priority),
03707    );
03708    if (!filtered.length)
03709      return `<section class="panel">${empty(changes.length ? "当前筛选下没有变更" :
03710   "两个版本未发现章节内容差异", changes.length ? "切换状态或优先级，查看其他变更。"
03711   : "已保留本次比对记录，可导出报告。", "", true)}</section>`;
03712    return filtered
03713      .map((change) => {
03714        const index = changes.findIndex((item) => item.id === change.id) + 1;
03715        const note = state.drafts.has(change.id)
03716          ? state.drafts.get(change.id)
03717          : change.note || "";
03718        return `<article class="change-card" id="change-${id(change.id)}" data-prior
03719  ity="${change.priority === "high" ? "high" : "normal"}"><header class="change-card
03720  -head"><div class="change-title"><span class="change-index">${String(index).padSta
03721  rt(2, "0")}</span><h3>${esc(change.section_title)}</h3>${kindTag(change.kind)}${ch
03722  ange.priority === "high" ? '<span class="tag amber">重点复核</span>' : ""}</div>${
03723  statusTag(change.status)}</header>${change.flags?.length ? `<div class="change-fla
03724  gs">${change.flags.map((flag) => `<span>${esc(flag)}</span>`).join("")}</div>` : "
03725  "}<div class="diff-grid"><div class="diff-column"><div class="diff-label"><span>旧
03726  版内容</span><span>${esc(change.old_location || "—")}</span></div><div class="diff
03727  -text">${diffText(change, "old")}</div></div><div class="diff-column"><div class="
03728  diff-label"><span>新版内容</span><span>${esc(change.new_location || "—")}</span></
03729  div><div class="diff-text">${diffText(change, "new")}</div></div></div><div class=
03730  "review-area">${change.reviewed_at ? `<div class="review-history"><strong>${esc(ch
03731  ange.reviewer || "本地审核员")}</strong> · ${esc(fmtDate(change.reviewed_at, true)
03732  )} · 已保存上次审核记录</div>` : ""}<div class="review-controls"><div class="revie
03733  w-note"><label for="note-${id(change.id)}">复核意见 <span class="muted">（忽略变更
03734  时必填）</span></label><textarea id="note-${id(change.id)}" data-note-id="${id(cha
03735  nge.id)}" maxlength="3000" rows="1" placeholder="记录变更依据或处理意见…">${esc(no
03736  te)}</textarea></div><div class="actions">${change.status !== "pending" ? button("
03737  重置待复核", "review", `data-id="${id(change.id)}" data-status="pending"`) : ""}${
03738  button("忽略", "review", `data-id="${id(change.id)}" data-status="ignored"`)}${but
03739  ton(change.status === "confirmed" ? "保存确认意见" : "确认变更", "review", `data-i
03740  d="${id(change.id)}" data-status="confirmed"`, true, "check")}</div></div></div></
```

源文件范围：app/static/app.js:523 → app/static/app.js:553

```
03741  article>`;
03742      })
03743      .join("");
03744  }
03745  async function auditPage() {
03746    state.audit = await api("/api/audit");
03747    return (
03748      pageHeading(
03749        "ACTIVITY TRAIL",
03750        "操作记录",
03751        "记录导入、章节校正、版本比对与每次审核结论的变化。",
03752        button("刷新记录", "refresh", "", "", "clock"),
03753      ) +
03754      `<section class="panel"><div class="toolbar"><div class="search-wrap">${icon("
03755  search")}<input id="audit-search" type="search" aria-label="搜索操作记录" placehol
03756  der="搜索操作、对象或记录内容"></div><span class="count-label" id="audit-count">${
03757  state.audit.length} 条记录</span></div><div id="audit-list">${auditTable(state.aud
03758  it)}</div></section>`
03759    );
03760  }
03761  function auditTable(items) {
03762    if (!items.length)
03763      return empty(
03764        "暂无匹配的操作记录",
03765        "建立药品、导入版本和审核变更后，将在此保留操作过程。",
03766        "",
03767        true,
03768      );
03769    return `<div class="table-scroll"><table><thead><tr><th>时间</th><th>操作</th><t
03770  h>操作对象</th><th>记录内容</th><th>记录人</th></tr></thead><tbody>${items.map((it
03771  em) => `<tr><td class="nowrap muted">${esc(fmtDate(item.created_at, true))}</td><t
03772  d class="nowrap">${esc(auditLabel(item.action))}</td><td><span class="tag gray">${
03773  esc({ drug: "药品", version: "版本", comparison: "比对", change: "变更" }[item.ent
03774  ity_type] || item.entity_type)} #${esc(item.entity_id)}</span></td><td class="audi
03775  t-detail">${esc(detailText(item.detail))}</td><td class="nowrap muted">${esc(item.
03776  actor)}</td></tr>`).join("")}</tbody></table></div>`;
03777  }
03778  function newDrugModal() {
03779    showModal(
03780      "建立药品档案",
03781      `<form id="drug-form"><div class="help-box">同一药品的不同说明书版本应放在同一
03782  档案中，便于准确比对。</div><div class="form-grid"><div class="field span-2"><labe
03783  l for="drug-name">药品名称 <span class="required">*</span></label><input id="drug-
03784  name" name="name" maxlength="200" required placeholder="例如：某药品片剂"></div><d
03785  iv class="field"><label for="drug-manufacturer">生产厂家</label><input id="drug-ma
03786  nufacturer" name="manufacturer" maxlength="200" placeholder="填写说明书标示厂家"><
03787  /div><div class="field"><label for="drug-specification">药品规格</label><input id=
03788  "drug-specification" name="specification" maxlength="200" placeholder="按说明书填
03789  写"></div><div class="field span-2"><label for="drug-approval">批准文号</label><in
03790  put id="drug-approval" name="approval_number" maxlength="100" placeholder="可选"><
03791  /div><div class="field span-2"><label for="drug-notes">备注</label><textarea id="d
03792  rug-notes" name="notes" maxlength="3000" placeholder="例如：资料用途、来源说明"></
03793  textarea></div></div><div class="form-actions">${button("取消", "close-modal")}<bu
03794  tton class="button primary" type="submit">${icon("plus")} 建立档案</button></div><
03795  /form>`,
```

源文件范围：app/static/app.js:554 → app/static/app.js:582

```
  );
}
function importModal(drugId) {
  showModal(
    "导入说明书版本",
    `<form id="import-form" data-drug-id="${id(drugId)}"><div class="form-grid"><d
iv class="field"><label for="version-label">版本名称 <span class="required">*</spa
n></label><input id="version-label" name="label" required maxlength="120" placehol
der="例如：2026 年修订版"></div><div class="field"><label for="version-date">版本
日期 <span class="required">*</span></label><input id="version-date" name="version
_date" type="date" required></div><div class="field span-2"><label for="version-so
urce">资料来源 <span class="required">*</span></label><input id="version-source" n
ame="source" required maxlength="1000" placeholder="文件出处、公告链接或资料提供方
"><small>来源信息会随比对报告保留，请填写可追溯的出处。</small></div></div><div cl
ass="segmented import-tabs" role="group" aria-label="导入方式"><button type="butto
n" class="active" data-action="import-tab" data-value="file" aria-pressed="true">$
{icon("upload")} 上传文件</button><button type="button" data-action="import-tab" d
ata-value="text" aria-pressed="false">${icon("edit")} 粘贴文本</button></div><inpu
t type="hidden" id="import-mode" value="file"><div id="import-file-pane" class="ta
b-pane"><div class="upload-area">${icon("upload")}<label for="version-file">选择说
明书文件</label><input id="version-file" name="file" type="file" required accept="
.pdf,.docx,.txt"><small>纯文字 PDF（不含图片）/ DOCX / TXT · 最大 10 MB</small></d
iv><p class="field-hint" style="margin-top:12px">含图片的 PDF（包括扫描件或带图片
页眉的文档）请先转换为经核对的 DOCX 或 TXT。导入后可手动校正章节识别结果。</p></di
v><div id="import-text-pane" class="tab-pane" hidden><div class="field"><label for
="version-text">说明书原文 <span class="required">*</span></label><textarea id="ve
rsion-text" name="text" disabled rows="10" placeholder="请保留章节标题，如：&#10;
【药品名称】&#10;…&#10;【用法用量】&#10;…"></textarea><small>请使用有权使用的资料
；模拟内容应明确标注。</small></div></div><div class="form-actions">${button("取消
", "close-modal")}<button class="button primary" type="submit">${icon("upload")}
导入并识别章节</button></div></form>`,
  );
}
async function newComparisonModal(drugId) {
  const versions = await api(`/api/drugs/${id(drugId)}/versions`);
  if (versions.length < 2) {
    toast("请先为该药品导入至少两个版本。", true);
    return;
  }
  const sorted = [...versions].sort(
    (a, b) =>
      String(a.version_date).localeCompare(String(b.version_date)) ||
      a.id - b.id,
  );
  const options = (selected) =>
    sorted
      .map(
        (version) =>
          `<option value="${id(version.id)}" ${version.id === selected ? "selected
" : ""}>${esc(version.label)} · ${esc(version.version_date)}</option>`,
      )
      .join("");
  showModal(
    "创建版本比对",
    `<form id="comparison-form"><div class="help-box">请选择同一药品的旧版与新版。
```

源文件范围：app/static/app.js:582 → app/static/app.js:600

```
  创建比对后，所用版本的章节内容将固定，后续修订请另行导入新版本。</div><div class="
  field"><label for="old-version">旧版说明书</label><select id="old-version" name="o
  ld_version_id" required>${options(sorted[sorted.length - 2].id)}</select></div><di
  v class="field"><label for="new-version">新版说明书</label><select id="new-version
  " name="new_version_id" required>${options(sorted[sorted.length - 1].id)}</select>
  </div><p class="field-hint">建议先查看章节识别结果，确认文档提取完整。新版日期不能
  早于旧版。</p><div class="form-actions">${button("取消", "close-modal")}<button ty
  pe="submit" class="button primary">${icon("compare")} 生成章节比对</button></div><
  /form>`,
    );
  }
  function sectionEditor(section, index) {
    return `<div class="section-editor"><div class="section-editor-head"><span class
  ="section-number">${String(index + 1).padStart(2, "0")}</span><input name="section
  _title" aria-label="第 ${index + 1} 个章节标题" required maxlength="200" value="${
  esc(section.title)}"><button class="icon-button" type="button" data-action="remove
  -section" aria-label="移除第 ${index + 1} 个章节">×</button></div><textarea name="
  section_text" rows="4" aria-label="第 ${index + 1} 个章节正文">${esc(section.text)
  }</textarea></div>`;
  }
  async function viewVersionModal(versionId) {
    const version = await api(`/api/versions/${id(versionId)}`);
    state.modalVersion = version;
    showModal(
      version.label,
      `<div class="version-meta"><span>版本日期 ${esc(version.version_date)}</span><
  span>${esc(version.filename || "粘贴文本")}</span><span>${version.sections?.length
   || 0} 个章节</span></div>${warnings(version.warnings)}<div class="segmented impor
  t-tabs" role="group" aria-label="版本查看方式"><button type="button" class="active
  " data-action="version-tab" data-value="sections" aria-pressed="true">章节与校正</
  button><button type="button" data-action="version-tab" data-value="raw" aria-press
  ed="false">原始提取文本</button><button type="button" data-action="version-tab" da
  ta-value="source" aria-pressed="false">来源与文件</button></div><div class="tab-pa
  ne" id="version-sections-pane"><div class="help-box">检查标题和正文是否完整。尚未
  参与比对的版本可以校正章节；已参与比对的版本会锁定，以保留原有审核依据。校正不会改
  写原始文件。</div><form id="sections-form" data-version-id="${id(versionId)}"><div
   id="section-editors">${(version.sections || []).map(sectionEditor).join("")}</div
  ><div class="editor-actions">${button("添加章节", "add-section", "", "", "plus")}<
  /div><div class="form-actions sticky-form-actions">${button("关闭", "close-modal")
  }<button type="submit" class="button primary">${icon("check")} 保存章节校正</butto
  n></div></form></div><div class="tab-pane" id="version-raw-pane" hidden><pre class
  ="raw-text">${esc(version.text)}</pre></div><div class="tab-pane" id="version-sour
  ce-pane" hidden><div class="field"><span class="metadata-label">资料来源</span><di
  v class="metadata-value">${esc(version.source || "未填写")}</div></div><div class=
  "field"><span class="metadata-label">原始文件 SHA-256</span><div class="hash">${es
  c(version.sha256)}</div></div><div class="field"><span class="metadata-label">导入
  时间</span><div class="metadata-value">${esc(fmtDate(version.created_at, true))}</
  div></div><a class="button" href="/api/versions/${id(version.id)}/file">${icon("do
  wnload")} 下载原始文件</a></div>`,
      true,
    );
    state.modalVersion = version;
  }
  document.addEventListener("input", (event) => {
    const element = event.target;
```

源文件范围：app/static/app.js:601 → app/static/app.js:655

```
    if (element.dataset.noteId) {
      state.drafts.set(Number(element.dataset.noteId), element.value);
      return;
    }
    if (element.id === "drug-search") {
      const query = element.value.toLocaleLowerCase().trim();
      const filtered = state.drugs.filter((item) =>
        [item.name, item.manufacturer, item.approval_number].some((value) =>
          String(value || "")
            .toLocaleLowerCase()
            .includes(query),
        ),
      );
      $("#drug-list").innerHTML = drugTable(filtered);
      $("#drug-count").textContent = `${filtered.length} 个药品档案`;
    }
    if (element.id === "comparison-search") filterComparisonTable();
    if (element.id === "audit-search") {
      const query = element.value.toLocaleLowerCase().trim();
      const filtered = state.audit.filter((item) =>
        [
          auditLabel(item.action),
          item.entity_type,
          item.entity_id,
          item.actor,
          detailText(item.detail),
        ].some((value) =>
          String(value || "")
            .toLocaleLowerCase()
            .includes(query),
        ),
      );
      $("#audit-list").innerHTML = auditTable(filtered);
      $("#audit-count").textContent = `${filtered.length} 条记录`;
    }
  });
  function filterComparisonTable() {
    const query = $("#comparison-search").value.toLocaleLowerCase().trim();
    const status = $("#comparison-status").value;
    const filtered = state.comparisons.filter(
      (item) =>
        [item.drug_name, item.old_label, item.new_label].some((value) =>
          String(value || "")
            .toLocaleLowerCase()
            .includes(query),
        ) &&
        (status === "all" ||
          (status === "pending"
            ? Number(item.pending_changes) > 0
            : !Number(item.pending_changes))),
    );
    $("#comparison-list").innerHTML = comparisonTable(filtered);
    $("#comparison-count").textContent = `${filtered.length} 个比对任务`;
  }
  document.addEventListener("change", (event) => {
```

源文件范围：app/static/app.js:656 → app/static/app.js:711

```
03961    if (event.target.id === "comparison-status") filterComparisonTable();
03962    if (event.target.id === "priority-filter") {
03963      state.priority = event.target.value;
03964      $("#changes-list").innerHTML = changesMarkup();
03965    }
03966  });
03967  document.addEventListener("click", async (event) => {
03968    const trigger = event.target.closest("[data-action]");
03969    if (!trigger || trigger.disabled) return;
03970    const action = trigger.dataset.action;
03971    try {
03972      if (action === "new-drug") return newDrugModal();
03973      if (action === "close-modal") return closeModal();
03974      if (action === "refresh") return navigate();
03975      if (action === "import-version") return importModal(trigger.dataset.id);
03976      if (action === "filter-review") {
03977        state.reviewStatus = trigger.dataset.value;
03978        $$('[data-action="filter-review"]').forEach((button) => {
03979          const active = button.dataset.value === state.reviewStatus;
03980          button.classList.toggle("active", active);
03981          button.setAttribute("aria-pressed", String(active));
03982        });
03983        $("#changes-list").innerHTML = changesMarkup();
03984        return;
03985      }
03986      if (action === "import-tab") {
03987        const mode = trigger.dataset.value;
03988        $("#import-mode").value = mode;
03989        $$('[data-action="import-tab"]').forEach((button) => {
03990          const active = button.dataset.value === mode;
03991          button.classList.toggle("active", active);
03992          button.setAttribute("aria-pressed", String(active));
03993        });
03994        $("#import-file-pane").hidden = mode !== "file";
03995        $("#import-text-pane").hidden = mode !== "text";
03996        $("#version-file").disabled = mode !== "file";
03997        $("#version-file").required = mode === "file";
03998        $("#version-text").disabled = mode !== "text";
03999        $("#version-text").required = mode === "text";
04000        return;
04001      }
04002      if (action === "version-tab") {
04003        const tab = trigger.dataset.value;
04004        $$('[data-action="version-tab"]').forEach((button) => {
04005          const active = button.dataset.value === tab;
04006          button.classList.toggle("active", active);
04007          button.setAttribute("aria-pressed", String(active));
04008        });
04009        ["sections", "raw", "source"].forEach((name) => {
04010          $(`#version-${name}-pane`).hidden = name !== tab;
04011        });
04012        return;
04013      }
04014      if (action === "add-section") {
04015        $("#section-editors").insertAdjacentHTML(
```

源文件范围：app/static/app.js:712 → app/static/app.js:767

```
          "beforeend",
          sectionEditor(
            { title: "新章节", text: "" },
            $$(".section-editor").length,
          ),
        );
        $$(".section-editor").at(-1).querySelector("input").focus();
        return;
      }
      if (action === "remove-section") {
        if ($$(".section-editor").length <= 1) {
          modalError("至少保留一个章节。");
          return;
        }
        const editor = trigger.closest(".section-editor");
        if (
          $("textarea", editor).value.trim() &&
          !window.confirm(
            "移除此章节会从结构化版本中删除其正文，原始文件仍保留。确认移除？",
          )
        )
          return;
        editor.remove();
        return;
      }
      trigger.disabled = true;
      if (action === "demo") {
        const data = await api("/api/demo", { method: "POST" });
        toast("模拟示例已就绪，所有示例内容均不用于临床。");
        const next = `comparisons/${id(data.comparison_id)}`;
        if (state.route === next) await navigate();
        else location.hash = next;
      } else if (action === "view-version")
        await viewVersionModal(trigger.dataset.id);
      else if (action === "new-comparison")
        await newComparisonModal(trigger.dataset.id);
      else if (action === "review") await saveReview(trigger);
    } catch (error) {
      if (modal.open) modalError(error.message);
      else toast(error.message, true);
    } finally {
      if (trigger.isConnected) trigger.disabled = false;
    }
  });
  async function saveReview(trigger) {
    const change = state.comparison.changes.find(
      (item) => item.id === Number(trigger.dataset.id),
    );
    if (!change) throw new Error("变更记录已更新，请刷新页面。");
    const note = $(`#note-${id(change.id)}`).value.trim();
    const status = trigger.dataset.status;
    if (status === "ignored" && !note) {
      $(`#note-${id(change.id)}`).focus();
      throw new Error("请填写忽略原因后再保存。");
    }
```

源文件范围：app/static/app.js:768 → app/static/app.js:822

```
    const card = $(`#change-${id(change.id)}`);
    const controls = $$("button, textarea", card);
    controls.forEach((element) => {
      element.disabled = true;
    });
    try {
      await api(`/api/changes/${id(change.id)}`, {
        method: "PATCH",
        body: { status, note, revision: change.revision },
      });
      state.drafts.delete(change.id);
      state.comparison = await api(`/api/comparisons/${id(state.comparison.id)}`);
      const scrollY = window.scrollY;
      main.innerHTML = comparisonMarkup();
      window.scrollTo(0, scrollY);
      toast("审核意见已保存，操作记录已更新。");
      const focusTarget =
        $(`#note-${id(change.id)}`) || $('[data-action="filter-review"].active');
      if (focusTarget) focusTarget.focus({ preventScroll: true });
    } catch (error) {
      if (error.status === 409) {
        state.comparison = await api(
          `/api/comparisons/${id(state.comparison.id)}`,
        );
        main.innerHTML = comparisonMarkup();
        throw new Error(
          "此变更已被其他页面更新，已载入最新状态并保留你的未提交意见。请核对后重新
保存。",
        );
      }
      throw error;
    } finally {
      controls.forEach((element) => {
        element.disabled = false;
      });
    }
  }
  document.addEventListener("submit", async (event) => {
    const form = event.target;
    if (
      !["drug-form", "import-form", "comparison-form", "sections-form"].includes(
        form.id,
      )
    )
      return;
    event.preventDefault();
    if (form.dataset.busy === "true") return;
    const submit = $('button[type="submit"]', form);
    const previousText = submit.innerHTML;
    const formData = new FormData(form);
    form.dataset.busy = "true";
    submit.disabled = true;
    submit.innerHTML = '<span class="spinner"></span> 正在处理…';
    $(".dialog-error", modal)?.remove();
    try {
```

源文件范围：app/static/app.js:823 → app/static/app.js:877

```
04126      if (form.id === "drug-form") {
04127        const payload = Object.fromEntries(formData);
04128        Object.keys(payload).forEach((key) => {
04129          payload[key] = payload[key].trim();
04130        });
04131        if (!payload.name) throw new Error("请输入药品名称。");
04132        const drug = await api("/api/drugs", { method: "POST", body: payload });
04133        closeModal();
04134        toast("药品档案已建立，现在可以导入说明书。");
04135        location.hash = `drugs/${id(drug.id)}`;
04136      } else if (form.id === "import-form") {
04137        const drugId = form.dataset.drugId;
04138        const mode = $("#import-mode").value;
04139        let version;
04140        if (mode === "file") {
04141          const file = formData.get("file");
04142          if (!file || !file.size) throw new Error("请选择非空的说明书文件。");
04143          if (file.size > 10 * 1024 * 1024)
04144            throw new Error("文件超过 10 MB，请缩减后重试。");
04145          if (!/\.(pdf|docx|txt)$/i.test(file.name))
04146            throw new Error("请选择 PDF、DOCX 或 TXT 文件。");
04147          version = await api(`/api/drugs/${id(drugId)}/versions`, {
04148            method: "POST",
04149            body: formData,
04150          });
04151        } else {
04152          const payload = {
04153            label: formData.get("label").trim(),
04154            version_date: formData.get("version_date"),
04155            source: formData.get("source").trim(),
04156            text: formData.get("text"),
04157          };
04158          if (!payload.text.trim()) throw new Error("请粘贴说明书原文。");
04159          version = await api(`/api/drugs/${id(drugId)}/versions/text`, {
04160            method: "POST",
04161            body: payload,
04162          });
04163        }
04164        closeModal();
04165        toast("版本已导入，请检查章节识别结果。");
04166        await navigate();
04167        await viewVersionModal(version.id);
04168      } else if (form.id === "comparison-form") {
04169        const oldId = Number(formData.get("old_version_id"));
04170        const newId = Number(formData.get("new_version_id"));
04171        if (oldId === newId) throw new Error("请选择两个不同的说明书版本。");
04172        const result = await api("/api/comparisons", {
04173          method: "POST",
04174          body: { old_version_id: oldId, new_version_id: newId },
04175        });
04176        closeModal();
04177        toast("比对已生成，开始逐条复核。");
04178        const route = `comparisons/${id(result.id)}`;
04179        if (state.route === route) await navigate();
04180        else location.hash = route;
```

源文件范围：app/static/app.js:878 → app/reports.py:37

```
      } else if (form.id === "sections-form") {
        const sections = $$(".section-editor", form).map((editor) => ({
          title: $("input", editor).value.trim(),
          text: $("textarea", editor).value,
        }));
        if (sections.some((section) => !section.title))
          throw new Error("每个章节都需要填写标题。");
        if (!sections.length || !sections.some((section) => section.text.trim()))
          throw new Error("请至少保留一个有正文的章节。");
        const versionId = form.dataset.versionId;
        await api(`/api/versions/${id(versionId)}/sections`, {
          method: "PUT",
          body: { sections },
        });
        toast("章节校正已保存，原始文件保持不变。");
        await navigate();
        await viewVersionModal(versionId);
      }
    } catch (error) {
      modalError(error.message);
    } finally {
      form.dataset.busy = "false";
      if (submit.isConnected) {
        submit.disabled = false;
        submit.innerHTML = previousText;
      }
    }
  });
  navigate();
  """Portable review reports. All human-supplied HTML is escaped on output."""
  from __future__ import annotations
  import csv
  from datetime import datetime, timezone
  from html import escape
  import io
  import json
  from typing import Any
  STATUSES = {"pending": "待复核", "confirmed": "已确认", "ignored": "已忽略"}
  KINDS = {"added": "新增章节", "removed": "删除章节", "modified": "内容修改"}
  PRIORITIES = {"high": "重点关注", "normal": "常规"}
  def _text(value: Any) -> str:
      if value is None:
          return ""
      if isinstance(value, (dict, list)):
          return json.dumps(value, ensure_ascii=False)
      return str(value)
  def _html(value: Any) -> str:
      return escape(_text(value), quote=True)
  def _csv_cell(value: Any) -> str:
      """Prevent spreadsheet applications interpreting untrusted cells as formulae.
      Leading whitespace can hide a formula marker, and tab/CR/LF themselves are
      unsafe leading characters in some spreadsheet importers. Prefix the entire
      original cell with a quote so the original content remains readable.
      """
      text = _text(value)
```

源文件范围：app/reports.py:38 → app/reports.py:91

```
    trimmed = text.lstrip(" \t\r\n\v\f\ufeff")
    if (text[:1] in ("\t", "\r", "\n")) or (
        trimmed and trimmed[0] in "=+-@"
    ):
        return "'" + text
    return text
def _provenance(version: dict, label: str) -> str:
    fields = (
        ("版本名称", version.get("label")),
        ("版本日期", version.get("version_date")),
        ("资料来源", version.get("source")),
        ("原始文件", version.get("filename")),
        ("导入时间", version.get("created_at")),
        ("SHA-256", version.get("sha256")),
    )
    rows = "".join(
        f"<dt>{_html(key)}</dt><dd>{_html(value) or '—'}</dd>"
        for key, value in fields
    )
    warnings = version.get("warnings") or []
    warning_html = "".join(f"<li>{_html(w)}</li>" for w in warnings)
    if warning_html:
        warning_html = f'<ul class="warning">{warning_html}</ul>'
    return f'<section class="version"><h3>{_html(label)}</h3><dl>{rows}</dl>{warni
ng_html}</section>'
def _change_html(change: dict, index: int) -> str:
    status = _text(change.get("status", "pending"))
    flags = change.get("flags") or []
    flag_html = "".join(f'<span class="flag">{_html(flag)}</span>' for flag in fla
gs)
    segments = []
    for segment in change.get("segments") or []:
        text = _html(segment.get("text", ""))
        op = segment.get("op")
        if op == "delete":
            segments.append(f"<del>{text}</del>")
        elif op == "insert":
            segments.append(f"<ins>{text}</ins>")
        else:
            segments.append(text)
    inline = (
        '<details><summary>逐字差异（删除线为旧内容，下划线为新内容）</summary>'
        f'<pre class="inline-diff">{"".join(segments)}</pre></details>'
        if segments else ""
    )
    return f"""
    <article class="change">
      <header class="change-title"><h3>{index:02d} · {_html(change.get('section_ti
tle', '未命名章节'))}</h3>
      <span class="badge">{_html(KINDS.get(change.get('kind'), change.get('kind',
'')))}</span>
      <span class="badge">{_html(PRIORITIES.get(change.get('priority'), change.get
('priority', '')))}</span>
      <span class="badge">{_html(STATUSES.get(status, status))}</span></header>
      <div class="flags">{flag_html}</div>
```

源文件范围：app/reports.py:92 → app/reports.py:127

```
        <div class="columns">
          <section><h4>旧版原文</h4><p class="location">{_html(change.get('old_locat
ion')) or '无对应章节'}</p>
            <pre>{_html(change.get('old_text')) or '（无）'}</pre></section>
          <section><h4>新版原文</h4><p class="location">{_html(change.get('new_locat
ion')) or '无对应章节'}</p>
            <pre>{_html(change.get('new_text')) or '（无）'}</pre></section>
        </div>
        {inline}
        <section class="review"><h4>人工审核记录</h4>
          <p>状态：{_html(STATUSES.get(status, status))}  ·  操作员：{_html(change.g
et('reviewer')) or '—'}  ·  记录修订号：{_html(change.get('revision', 0))}</p>
          <p>复核时间：{_html(change.get('reviewed_at')) or '尚未复核'}</p>
          <p class="note">{_html(change.get('note')) or '未填写审核意见'}</p>
        </section>
      </article>"""
def _html_report(comparison: dict, exported_at: str) -> bytes:
      changes = comparison.get("changes") or []
      counts = {key: sum(c.get("status", "pending") == key for c in changes) for key
 in STATUSES}
      old = comparison.get("old_version") or {}
      new = comparison.get("new_version") or {}
      warnings = "".join(f"<li>{_html(w)}</li>" for w in comparison.get("warnings")
or [])
      warning_block = f'<aside class="warning"><strong>比对提示</strong><ul>{warning
s}</ul></aside>' if warnings else ""
      blocks = "".join(_change_html(c, i + 1) for i, c in enumerate(changes))
      if not blocks:
          blocks = '<section class="empty">未检出章节内容差异。此结果不等同于临床或
法规一致性结论。</section>'
      html = f"""<!doctype html>
<html lang="zh-CN"><head><meta charset="UTF-8"><meta name="viewport" content="widt
h=device-width,initial-scale=1">
<meta http-equiv="Content-Security-Policy" content="default-src 'none'; style-src
'unsafe-inline'; img-src data:; base-uri 'none'; form-action 'none'">
<title>说明书变更审核报告 #{_html(comparison.get('id'))}</title>
<style>
*{{box-sizing:border-box}}body{{margin:0;background:#f3f6f9;color:#1c3042;font:14p
x/1.7 system-ui,-apple-system,"Microsoft YaHei",sans-serif}}
main{{max-width:1150px;margin:32px auto;background:white;padding:40px;border-radiu
s:16px}}h1{{margin:0 0 10px;font-size:28px;line-height:1.4}}h2{{margin:28px 0 14px
;font-size:20px}}h3,h4{{margin:0 0 8px}}
.eyebrow{{color:#087f85;font-size:12px;letter-spacing:2px}}.muted,.location{{color
:#5e6f7c}}.location{{font-size:12px;margin-top:0}}
.stats,.columns,.versions{{display:grid;grid-template-columns:repeat(2,minmax(0,1f
r));gap:18px}}.stats{{grid-template-columns:repeat(4,minmax(0,1fr));margin:25px 0}
}
.stat,.version{{padding:18px;border:1px solid #dce5ec;border-radius:10px}}.stat st
rong{{display:block;font-size:26px;color:#087f85}}dl{{display:grid;grid-template-c
olumns:76px minmax(0,1fr);gap:6px 10px;margin:0}}dt{{color:#5e6f7c}}dd{{margin:0;o
verflow-wrap:anywhere}}
.change{{border:1px solid #dce5ec;border-radius:10px;margin:18px 0;padding:22px}}.
change-title{{display:flex;gap:8px;align-items:center;flex-wrap:wrap}}.change-titl
e h3{{margin-right:auto}}.badge,.flag{{display:inline-block;border-radius:4px;padd
ing:2px 7px;font-size:12px;background:#eaf2f7}}.flag{{background:#fff0d4;color:#78
```

源文件范围：app/reports.py:127 → app/reports.py:157

```
510a;margin:0 5px 8px 0}}.flags{{margin-top:8px}}
pre{{font:inherit;font-size:13px;white-space:pre-wrap;overflow-wrap:anywhere;backg
round:#f6f8fa;padding:14px;border-radius:6px;margin:0}}.review{{margin-top:16px;bo
rder-top:1px solid #dce5ec;padding-top:14px}}.review p{{margin:5px 0}}.note{{white
-space:pre-wrap;overflow-wrap:anywhere}}details{{margin-top:14px}}summary{{cursor:
pointer;color:#087f85}}del{{background:#fee4e4;text-decoration:line-through}}ins{{
background:#d9f3e4;text-decoration:underline}}.warning{{background:#fff7e9;padding
:14px;border-radius:8px}}.empty{{padding:28px;border:1px dashed #a8c4d2;border-rad
ius:8px}}footer{{margin-top:30px;border-top:1px solid #dce5ec;padding-top:15px;col
or:#5e6f7c;font-size:12px}}.print-tip{{font-size:12px;color:#5e6f7c}}
@media(max-width:700px){{main{{padding:20px;margin:0;border-radius:0}}.columns,.ve
rsions{{grid-template-columns:1fr}}.stats{{grid-template-columns:repeat(2,1fr)}}}}
@media print{{@page{{size:A4;margin:15mm}}body{{background:white;font-size:10pt}}m
ain{{padding:0;margin:0;max-width:none}}.print-tip{{display:none}}.change{{break-i
nside:auto;border-radius:0}}.change-title,.review{{break-inside:avoid}}pre{{font-s
ize:9pt}}a{{color:inherit}}details{{display:none}}}}
</style></head><body><main>
<div class="eyebrow">MEDLABEL REVIEW · V1.0</div>
<h1>药品说明书版本比对与变更审核报告</h1>
<p><strong>{_html(comparison.get('drug_name', ''))}</strong> · 比对任务 #{_html(co
mparison.get('id'))}</p>
<p class="muted">任务创建：{_html(comparison.get('created_at'))} ｜ 导出时间：{_
html(exported_at)}（UTC）</p>
<p class="print-tip">可使用浏览器“打印 / 另存为 PDF”保存此报告。</p>
<div class="stats"><div class="stat"><strong>{len(changes)}</strong>变更章节</div>
<div class="stat"><strong>{counts['pending']}</strong>待复核</div><div class="stat
"><strong>{counts['confirmed']}</strong>已确认</div><div class="stat"><strong>{cou
nts['ignored']}</strong>已忽略</div></div>
<p class="muted">未变更章节：{_html(comparison.get('unchanged_sections', 0))}。本
报告反映导出时点的审核状态。</p>
<h2>资料来源与版本追溯</h2><div class="versions">{_provenance(old, '旧版')}{_prove
nance(new, '新版')}</div>
{warning_block}<h2>章节差异与审核意见</h2>{blocks}
<footer>药品说明书版本比对与变更审核系统 V1.0。此工具用于文本比对与记录整理，不判
断临床正确性或替代专业审核。操作员名称是本地配置的记录标签，未经身份认证；本报告不
构成电子签名。行号指向提取文本，手工校正章节的位置应结合原始文件复核。文件 SHA-256
 用于核对原始文件内容。</footer>
</main></body></html>"""
    return html.encode("utf-8")
def _csv_report(comparison: dict, exported_at: str) -> bytes:
    stream = io.StringIO(newline="")
    writer = csv.writer(stream)
    writer.writerow([
        "任务 ID", "药品名称", "导出时间 UTC", "旧版名称", "旧版日期", "旧版来源", "
旧版文件", "旧版 SHA256",
        "新版名称", "新版日期", "新版来源", "新版文件", "新版 SHA256", "章节", "变
化类型", "关注级别",
        "旧版位置", "新版位置", "旧版内容", "新版内容", "关注标记", "审核状态", "
审核意见", "本地操作员（未认证）", "复核时间", "修订号",
    ])
    old, new = comparison.get("old_version") or {}, comparison.get("new_version")
or {}
    common = [comparison.get("id"), comparison.get("drug_name"), exported_at]
    for version in (old, new):
        common.extend(version.get(key) for key in ("label", "version_date", "sourc
```

源文件范围：app/reports.py:157 → app/demo.py:18

```
e", "filename", "sha256"))
    changes = comparison.get("changes") or []
    if not changes:
        writer.writerow([_csv_cell(value) for value in common + ["未检出章节内容差
异"] + [""] * 12])
    for change in changes:
        row = common + [
            change.get("section_title"), KINDS.get(change.get("kind"), change.get(
"kind")),
            PRIORITIES.get(change.get("priority"), change.get("priority")),
            change.get("old_location"), change.get("new_location"), change.get("ol
d_text"), change.get("new_text"),
            "; ".join(_text(flag) for flag in change.get("flags") or []),
            STATUSES.get(change.get("status"), change.get("status")), change.get("
note"),
            change.get("reviewer"), change.get("reviewed_at"), change.get("revisio
n"),
        ]
        writer.writerow([_csv_cell(value) for value in row])
    return stream.getvalue().encode("utf-8-sig")
def export_report(comparison: dict, format: str) -> tuple[bytes, str, str]:
    """Return a ready-to-download body, content type, and safe ASCII filename."""
    exported_at = datetime.now(timezone.utc).isoformat(timespec="seconds")
    identifier = str(comparison.get("id", "report"))
    safe_id = "".join(char for char in identifier if char.isascii() and (char.isal
num() or char in "-_")) or "report"
    if format == "html":
        return _html_report(comparison, exported_at), "text/html; charset=utf-8",
f"comparison-{safe_id}.html"
    if format == "csv":
        return _csv_report(comparison, exported_at), "text/csv; charset=utf-8", f"
comparison-{safe_id}.csv"
    if format == "json":
        payload = dict(comparison)
        payload["exported_at"] = exported_at
        return json.dumps(payload, ensure_ascii=False, indent=2).encode("utf-8"),
"application/json; charset=utf-8", f"comparison-{safe_id}.json"
    raise ValueError("不支持的导出格式，请选择 html、csv 或 json。")
"""Fictional demonstration material, never real prescribing information."""
DRUG = {
    "name": "示例药品 A（模拟）",
    "manufacturer": "虚构教学制药公司",
    "specification": "仅用于软件演示",
    "approval_number": "SIMULATED-NOT-APPROVED",
    "notes": "模拟示例，不用于临床。所有名称、数据和变更均为虚构。",
}
OLD_TEXT = """模拟示例，不用于临床。以下为虚构说明书，用于演示文本比对。
【药品名称】
示例药品 A（模拟）
【成份】
虚构成份甲；本示例不对应任何真实药物。
【性状】
演示用白色片状物。
【适应症】
仅用于演示说明书软件，不具有任何适应症。
```

源文件范围：app/demo.py:19 → app/demo.py:54

```
04456  【用法用量】
04457  测试字段：数值 10 mg。此字段为虚构测试数据，禁止据此用药。
04458  【禁忌】
04459  演示规则甲：禁止用于临床。
04460  【不良反应】
04461  无真实临床数据；待补充演示字段。
04462  【贮藏】
04463  测试温度字段：20 ℃。此字段不构成储存指导。
04464  【包装】
04465  模拟包装 A。
04466  【说明书修订日期】
04467  2026-01-01
04468  """
04469  NEW_TEXT = """模拟示例，不用于临床。以下为虚构说明书，用于演示文本比对。
04470  【药品名称】
04471  示例药品 A（模拟）
04472  【成份】
04473  虚构成份甲；本示例不对应任何真实药物。
04474  【性状】
04475  演示用白色片状物。
04476  【适应症】
04477  仅用于演示说明书软件，不具有任何适应症。
04478  【用法用量】
04479  测试字段：数值 15 mg。此字段为虚构测试数据，禁止据此用药。
04480  【禁忌】
04481  演示规则甲：禁止用于临床。新增演示规则乙：不得替代药师审核。
04482  【不良反应】
04483  无真实临床数据；新增虚构字段用于演示变化定位。
04484  【注意事项】
04485  请由人工审核软件识别出的差异；本材料为模拟示例，不用于临床。
04486  【贮藏】
04487  测试温度字段：20 ℃。此字段不构成储存指导。
04488  【说明书修订日期】
04489  2026-06-01
04490  """
```